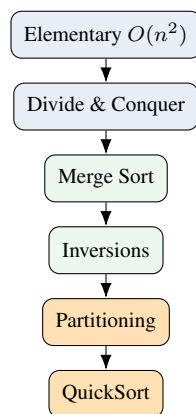


Sorting Algorithms & Partitioning

From Elementary Sorting to Merge Sort, QuickSort,
Partitioning, and Inversions



Java • LeetCode-oriented

Derive the algorithms from their invariants — don't memorise the code.

Contents

1	Learning Objectives and Prerequisites	1
1.1	What you should be able to do afterwards	1
1.2	The progression	1
1.3	Prerequisites	2
1.4	How to use this booklet	2
2	Sorting Fundamentals	4
2.1	What sorting means	4
2.2	In-place vs out-of-place	4
2.3	Stable vs unstable	5
2.4	Adaptive vs non-adaptive	5
2.5	Comparison-based vs non-comparison-based	5
2.6	The comparison table	6
3	The Three $O(n^2)$ Elementary Sorts	8
3.1	Bubble Sort	8
3.1.1	Intuition	8
3.1.2	Invariant	8
3.1.3	Example: [5, 3, 8, 1, 2]	8
3.1.4	Algorithm	9
3.1.5	Java implementation	9
3.1.6	Line by line	10
3.1.7	Complexity	10
3.1.8	When to use it	10
3.2	Selection Sort	10
3.2.1	Intuition	11
3.2.2	Invariant	11
3.2.3	Example: [5, 3, 8, 1, 2]	11
3.2.4	Java implementation	11
3.2.5	Line by line	12
3.2.6	Why it stays $O(n^2)$ even on sorted input	12
3.2.7	Why the number of swaps is small	12
3.2.8	Complexity	12
3.2.9	When to use it	12
3.3	Insertion Sort	13
3.3.1	Intuition	13
3.3.2	Invariant	13
3.3.3	Example: [5, 3, 8, 1, 2]	13
3.3.4	Swapping vs shifting	14
3.3.5	Java implementation	14
3.3.6	Line by line	15
3.3.7	Why the best case is $O(n)$	15
3.3.8	Complexity	15
3.3.9	When to use it	15

4	Inversions — What Sorting Actually Removes	17
4.1	Definition	17
4.2	Worked examples	17
4.3	The maximum number of inversions	18
4.4	Why inversions matter	18
4.4.1	Insertion Sort: a machine for removing inversions	18
4.4.2	Bubble Sort: one adjacent swap, one inversion	19
4.4.3	Selection Sort: the exception that proves the rule	19
4.5	Inversions as a problem-solving tool	19
5	Merge Sort	21
5.1	Start with the merge, not the recursion	21
5.1.1	The two-pointer merge	21
5.2	From merge to Merge Sort: divide and conquer	22
5.2.1	Why $O(n \log n)$ — the intuitive derivation	22
5.3	Top-down (recursive) Merge Sort	22
5.3.1	Line by line	23
5.3.2	Full recursion trace: [38, 27, 43, 3]	24
5.3.3	Complexity	24
5.3.4	When to use it	25
6	Bottom-Up (Iterative) Merge Sort	26
6.1	The idea: stop dividing, start doubling	26
6.2	Full trace on an odd-sized array	26
6.3	How incomplete final blocks are handled	27
6.4	Java implementation	28
6.5	Why <code>width *= 2</code> ?	28
6.6	Why the condition is <code>width < n</code> , not <code>width < n - 1</code>	29
6.7	Top-down vs bottom-up	29
7	Counting Inversions with Merge Sort	31
7.1	The problem	31
7.2	Counting cross inversions during the merge	31
7.3	Why the total is $O(n \log n)$	32
7.4	Java implementation	32
7.4.1	Line by line	34
7.4.2	Dry run on [2, 4, 1, 3, 5]	34
7.4.3	Complexity	35
8	QuickSort and Partitioning	36
8.1	The core idea: partition first	36
8.2	QuickSort vs Merge Sort: where the work happens	36
8.3	Lomuto Partition	36
8.3.1	Intuition	37
8.3.2	Invariant	37
8.3.3	Java implementation	37
8.3.4	Line by line	38
8.3.5	Manual trace: [4, 2, 7, 3, 1, 5], <code>pivot = 5</code>	38
8.3.6	Complexity	38
8.4	Hoare Partition	39
8.4.1	Intuition	39
8.4.2	Invariant	39

8.4.3	Java implementation	39
8.4.4	Line by line	40
8.4.5	Trace: <code>[4, 2, 7, 3, 1, 5]</code> , <code>pivot = arr[0] = 4</code>	40
8.5	The critical difference: what does the partition return?	41
8.6	Recursive QuickSort	41
8.6.1	With Lomuto	41
8.6.2	With Hoare	41
8.6.3	Why the ranges differ — carefully	42
8.7	Worst case, and how to avoid it	42
8.8	Complexity summary	43
9	Iterative QuickSort and the Explicit Stack	44
9.1	What recursion actually is	44
9.2	The stack as a pair of ints	44
9.3	Iterative QuickSort with Lomuto	44
9.3.1	Line by line	45
9.3.2	Trace on <code>[4, 2, 7, 3, 1, 5]</code>	46
9.4	Iterative QuickSort with Hoare	46
9.5	How big must the stack be?	46
9.6	Recursive vs iterative, across all five algorithms	47
9.6.1	When is recursion worth it?	48
10	Partitioning Beyond QuickSort	49
10.1	Partitioning is an idea, not a subroutine	49
10.2	Sorting vs partitioning vs selection	49
10.2.1	Selection: Quickselect	50
10.3	Three-way partitioning — the Dutch National Flag	50
10.3.1	The problem with two-way partitioning	50
10.3.2	LeetCode 75 — Sort Colors	50
10.3.3	Trace on <code>[2, 0, 2, 1, 1, 0]</code>	51
10.3.4	Three-way partitioning inside QuickSort	52
10.4	General 2-way partitioning	52
11	LeetCode Roadmap and Pattern Recognition	54
11.1	Beginner	54
11.2	Intermediate	54
11.3	Suggested order	56
11.4	How do I know which sorting technique to use?	56
11.4.1	Decision tree	57
12	Common Bugs and How to Find Them	58
12.1	Symptom table	58
12.2	Off-by-one in loop bounds	58
12.3	The <code>mid</code> calculation	58
12.4	Incorrect merge boundaries	59
12.5	Forgetting the drain loops	59
12.6	The incomplete final block in bottom-up Merge Sort	59
12.7	<code>width *= 2</code> vs <code>width++</code>	60
12.8	Lomuto vs Hoare recursive boundaries	60
12.9	Infinite recursion in QuickSort	60
12.10	Incorrect <code>i/j</code> initialisation in Hoare	61
12.11	<code><</code> vs <code><=</code> : stability and duplicates	61

12.12A debugging routine	62
13 Master Cheat Sheet	63
13.1 The five mechanisms	63
13.2 Quick reference	63
13.3 Complexity table	65
13.4 The distinctions that get tested	65
13.5 Can You Reconstruct It?	66

Chapter 1

Learning Objectives and Prerequisites

1.1 What you should be able to do afterwards

This booklet is not a reference sheet. Its purpose is that you can *rebuild* each algorithm from a one-sentence idea, because that is what actually survives interview pressure. Concretely, after working through it you should be able to:

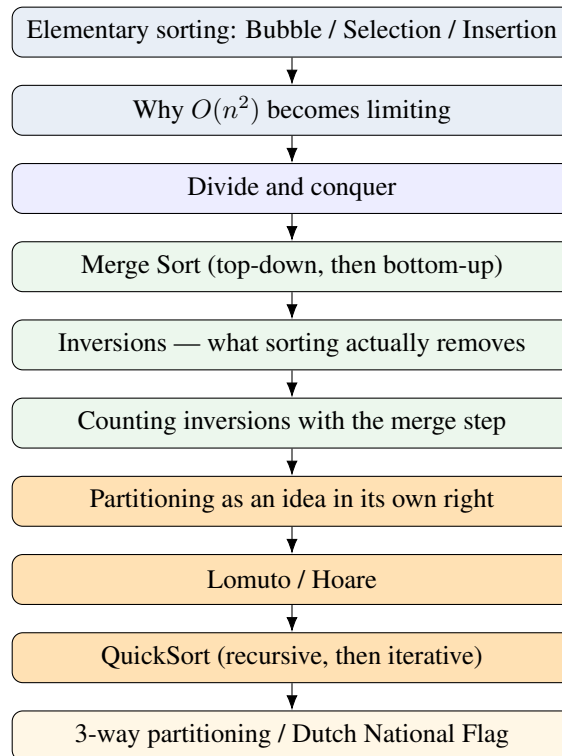
1. Recognise when sorting is the right tool for a problem — and when it is a detour.
2. Reconstruct Bubble, Selection and Insertion Sort from their invariants, without having memorised a single line.
3. Explain *why* Merge Sort is $O(n \log n)$, not merely state it.
4. Write both top-down recursive and bottom-up iterative Merge Sort, and say what the bottom-up version replaces recursion *with*.
5. Explain QuickSort as “partition, then recurse” rather than as a block of code.
6. Implement and distinguish Lomuto and Hoare partitioning — in particular, what each one *returns*.
7. Convert recursive QuickSort into iterative QuickSort with an explicit stack.
8. Define an inversion and count the inversions of a small array by hand.
9. Explain why inversions connect Insertion Sort and Merge Sort.
10. Use inversion counting as a general problem-solving technique.
11. Distinguish sorting, partitioning and selection, and pick the cheapest one that solves the problem.
12. Recognise the standard LeetCode patterns built on all of the above.

Key Insight

There are only **five mechanisms** in this entire booklet: adjacent-compare-and-swap (Bubble), find-minimum-and-swap (Selection), key-shift-insert (Insertion), merge-two-sorted-ranges (Merge), and rearrange-around-a-pivot (Partition). Everything else is one of these mechanisms plus either *recursion* or *an explicit loop over block sizes / an explicit stack*. If you only remember one page of this book, remember that sentence.

1.2 The progression

Each chapter exists because the previous one ran into a wall. Keep this map in view; the booklet returns to it repeatedly.



1.3 Prerequisites

You need very little, and this booklet will not re-teach it:

Arrays and indices	<code>arr[i]</code> , zero-based indexing, <code>arr.length</code> . You must be comfortable with a <i>range</i> written as $[l, r]$ meaning indices l through r <i>inclusive</i> — almost every function in this booklet takes such a range.
Loops	<code>for</code> and <code>while</code> , including nested loops and the idea that the inner loop bound may depend on the outer index.
Recursion	A function calling itself, a base case, and the fact that each call gets its own copy of local variables. Chapter 9 makes the call stack explicit, so you should at least believe it exists.
Divide and conquer	Split the problem, solve the parts, combine. That is all — Merge Sort will supply the intuition.
Complexity	$O(1)$, $O(n)$, $O(n \log n)$, $O(n^2)$ and the idea that we count operations as a function of input size, ignoring constants.

Interview Tip

One convention worth fixing now, because half of all sorting bugs come from mixing them up: in this booklet a range `(arr, l, r)` is always **inclusive on both ends**. So a range of length 1 is `l == r`, and the empty/finished case is `l >= r`. Java's own library uses half-open ranges (`Arrays.sort(a, from, to)` excludes `to`); pick one convention per function and write it in a comment.

1.4 How to use this booklet

1. Read the **Intuition** box first and try to write the algorithm yourself.

2. Read the **Invariant** box. If you can state the invariant, you can usually recover the loop bounds by asking “what must be true for the invariant to still hold?”
3. Work the traced example *on paper* before reading the code.
4. Read the code and the line-by-line notes.
5. Finish with the **Common Mistakes** box — these are the specific ways your implementation will fail, and recognising the symptom saves debugging time.
6. Use the “Can You Reconstruct It?” checklist in Chapter 13 as a self-test the night before an interview.

Chapter 2

Sorting Fundamentals

2.1 What sorting means

Sorting rearranges the elements of a collection into a specified order. Formally, given an array `arr` of length n , a sort produces a *permutation* of `arr` such that

$$\text{arr}[0] \leq \text{arr}[1] \leq \dots \leq \text{arr}[n-1].$$

Two words in that sentence carry weight. *Permutation* means no element is invented or lost — a “sort” that returns `[1, 1, 1]` for input `[3, 1, 2]` is not a sort. And $\leq$ (rather than $<$) means duplicates are allowed and must be kept.

Ascending vs descending

Ascending is $\leq$; descending is $\geq$. Every algorithm in this booklet becomes its descending twin by flipping a single comparison operator:

	Ascending	Descending
Bubble	if (<code>arr[j] > arr[j+1]</code>) swap	if (<code>arr[j] < arr[j+1]</code>) swap
Selection	if (<code>arr[j] < arr[minIdx]</code>)	if (<code>arr[j] > arr[maxIdx]</code>)
Insertion	while (<code>arr[j] > key</code>)	while (<code>arr[j] < key</code>)
Merge	if (<code>arr[i] <= arr[j]</code>)	if (<code>arr[i] >= arr[j]</code>)

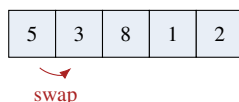
Interview Tip

Never write a second descending implementation. Sort ascending and reverse ($O(n)$), or — for objects — pass a reversed comparator. Interviewers read a duplicated descending routine as a sign you are copying rather than thinking.

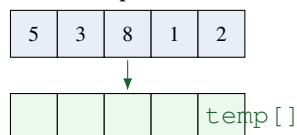
2.2 In-place vs out-of-place

An **in-place** sort uses $O(1)$ (or $O(\log n)$ for the recursion stack) auxiliary memory beyond the input array; it rearranges elements by swapping. An **out-of-place** sort allocates extra memory proportional to n .

In-place



Out-of-place



Bubble, Selection, Insertion and QuickSort are in-place. Standard Merge Sort is not: its merge step needs a temporary buffer, because merging two sorted halves into the same array would overwrite elements you have not yet read. (In-place merging exists but is complicated and slow — do not offer it in an interview unless asked.)

2.3 Stable vs unstable

A sort is **stable** if elements that compare equal keep their original relative order.

This only matters when elements carry more information than the sort key. Suppose we sort records by grade:

Input	A:2	B:1	C:2	D:1	
Stable	B:1	D:1	A:2	C:2	stable keeps B before D and A before C
Unstable	D:1	B:1	C:2	A:2	

Stability is what lets you sort by several keys in sequence: sort by secondary key first, then by primary key with a *stable* sort, and the secondary order survives inside each primary group.

Key Insight

Stability is destroyed by **long-distance swaps**. Bubble and Insertion only ever move an element past elements it is strictly out of order with, so they are stable. Selection Sort swaps a far-away minimum into position, jumping over equal elements — unstable. QuickSort’s partition swaps across the array — unstable. Merge Sort merges two adjacent runs and breaks ties in favour of the left run — stable, *provided* you write `arr[i] <= arr[j]` and not `<`.

2.4 Adaptive vs non-adaptive

An **adaptive** sort runs faster on input that is already partially ordered.

Algorithm	Sorted input	Adaptive?
Bubble (with swapped flag)	$O(n)$ — one clean pass, then break	Yes
Bubble (without the flag)	$O(n^2)$	No
Selection	$O(n^2)$ — must still scan for each minimum	No
Insertion	$O(n)$ — inner <code>while</code> never runs	Yes (strongly)
Merge	$O(n \log n)$ regardless	No
Quick	$O(n^2)$ with a naive pivot — <i>worse</i> on sorted input	No

Adaptivity is exactly the inversion story of Chapter 4: an adaptive sort’s running time depends on *how out of order* the input is, not just on n .

2.5 Comparison-based vs non-comparison-based

A **comparison sort** only ever asks “is $a < b$?”. Every algorithm in this booklet is a comparison sort. There is a famous lower bound:

Key Insight

Any comparison-based sort needs $\Omega(n \log n)$ comparisons in the worst case. *Why*: there are $n!$ possible orderings, each comparison has 2 outcomes, so a decision tree of depth d distinguishes at most 2^d orderings. We need $2^d \geq n!$, so $d \geq \log_2(n!) = \Theta(n \log n)$ by Stirling. This is why nobody has ever shown you an $O(n)$ comparison sort — none exists.

Non-comparison sorts escape the bound by exploiting the structure of the keys instead of comparing them: Counting Sort ($O(n + k)$ for integers in a small range $[0, k)$), Radix Sort, Bucket Sort. They are out of scope here, but keep them in mind for the pattern question “is the input constrained to a small range?” (Chapter 11).

2.6 The comparison table

Algorithm	Best	Average	Worst	Space	Stable	In-place
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No
QuickSort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes

Explaining every entry

Bubble — best $O(n)$. Only with the `swapped` flag. On sorted input the first pass performs $n - 1$ comparisons, no swaps, and breaks. Without the flag it is $O(n^2)$ always.

Bubble — average/worst $O(n^2)$. Pass i does $n - 1 - i$ comparisons, so $\sum_{i=0}^{n-2} (n - 1 - i) = \frac{n(n-1)}{2}$.

Bubble — stable. It only swaps *strictly* out-of-order neighbours ($>$ not $\geq$), so equal elements never cross.

Selection — best = worst = $O(n^2)$. The inner loop scans the whole unsorted suffix to find the minimum, regardless of the data. There is no early exit, because you cannot know an element is the minimum without looking at all of them. Hence non-adaptive.

Selection — unstable. The swap moves `arr[i]` to a far position, possibly jumping it past an equal element. Example: $[2a, 2b, 1]$. The minimum is 1 at index 2; swapping with index 0 gives $[1, 2b, 2a]$ — $2b$ now precedes $2a$.

Selection — few swaps. Exactly $n - 1$ swaps, i.e. $O(n)$ writes, versus $O(n^2)$ for Bubble. That matters when a write is far more expensive than a read (flash memory, EEPROM). It is Selection Sort’s only real advantage.

Insertion — best $O(n)$. On sorted input the guard `arr[j] > key` fails immediately every time, so the inner `while` body never executes: $n - 1$ comparisons total.

Insertion — worst $O(n^2)$. On reverse-sorted input the key must travel all the way to the front: $\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$ shifts.

Merge — $O(n \log n)$ in all cases. The recursion always splits in half, so the tree is always $\log_2 n$ deep, and every level does $O(n)$ merging work. Nothing in the data changes that — which is why it is both non-adaptive and reliably fast.

Merge — space $O(n)$. The temporary buffer. Add $O(\log n)$ for the recursion stack (dominated by the $O(n)$).

QuickSort — average $O(n \log n)$. A random pivot splits the array into pieces of expected constant ratio, giving $O(\log n)$ expected depth with $O(n)$ partitioning work per level.

QuickSort — worst $O(n^2)$. If every pivot is the minimum or maximum, each partition removes only one element and the depth becomes n . With `pivot = arr[r]` this happens on *already sorted* input — a genuinely nasty trap.

QuickSort — space $O(\log n)$. No buffer, but the recursion stack is $O(\log n)$ on average and $O(n)$ in the worst case (fixable by recursing into the smaller side first).

Common Mistake

Writing “QuickSort is $O(n \log n)$ ” without qualification. It is $O(n^2)$ in the worst case, and a naive last-element pivot triggers that worst case on sorted input — the most common real-world input shape. Always say “average $O(n \log n)$, worst $O(n^2)$ ” and mention pivot randomisation.

Chapter 3

The Three $O(n^2)$ Elementary Sorts

All three keep a *sorted region* and an *unsorted region*, and grow the sorted region by one element per outer iteration. They differ only in *where* the sorted region is and *how* the next element gets into it.

	Sorted region	How it grows
Bubble	suffix (right end)	the largest element bubbles right via adjacent swaps
Selection	prefix (left end)	scan the unsorted part for the minimum, swap it in
Insertion	prefix (left end)	take the next element, shift bigger ones right, drop it in

Key Insight

Bubble and Selection put the *right element in the right place*: after step i , one more element is in its *final* position. Insertion Sort does the opposite — it puts the *next element* in the right place *relative to what it has seen so far*; those elements may still all move later. This is why Insertion Sort is the one that adapts to nearly-sorted input, and the one that generalises to streaming data.

3.1 Bubble Sort

3.1.1 Intuition

Intuition

Walk along the array comparing each element with its right-hand neighbour. Whenever a pair is in the wrong order, swap them. A large element, once touched, keeps being swapped rightwards on the same pass — it “bubbles” to the end. After one full pass the largest element has necessarily reached the last position, because nothing can stop it: every comparison it takes part in sends it further right.

3.1.2 Invariant

Invariant

After outer pass i (0-indexed), the last $i + 1$ elements are the $i + 1$ largest elements of the array, in their final sorted positions.

$$\underbrace{\text{[unsorted]}}_{\text{indices } 0..n-2-i} \quad | \quad \underbrace{\text{[sorted, final]}}_{\text{indices } n-1-i..n-1}$$

Consequently pass i need only compare up to index $n - 2 - i$ — everything beyond is already done. That is exactly where the loop bound `j < arr.length - 1 - i` comes from; you never have to memorise it.

3.1.3 Example: [5, 3, 8, 1, 2]

Green cells are finalised; the bracket marks the pair being compared.

Pass 1 (compare indices 0–3; 4 comparisons)

```
[5, 3, 8, 1, 2]  5 > 3  swap
[3, 5, 8, 1, 2]  5 < 8  keep
[3, 5, 8, 1, 2]  8 > 1  swap
[3, 5, 1, 8, 2]  8 > 2  swap
[3, 5, 1, 2, 8]  8 is final
```

Pass 2 (compare indices 0–2)

```
[3, 5, 1, 2 | 8]  3 < 5  keep
[3, 5, 1, 2 | 8]  swap
[3, 1, 5, 2 | 8]  swap
[3, 1, 2, 5, 8]  5 is final
```

Pass 3 (indices 0–1)

```
[3, 1, 2 | 5, 8]  swap
[1, 3, 2 | 5, 8]  swap
[1, 2, 3, 5, 8]  3 is final
```

Pass 4 (index 0 only) — $1 < 2$, no swap. The swapped flag stays false, so we break. Result $[1, 2, 3, 5, 8]$.

pass 0	5	3	8	1	2
pass 1	3	5	1	2	8
pass 2	3	1	2	5	8
pass 3	1	2	3	5	8
pass 4	1	2	3	5	8

green = finalised suffix

3.1.4 Algorithm

1. For each pass i from 0 to $n - 2$:
2. Set swapped = false.
3. For each j from 0 to $n - 2 - i$: if $\text{arr}[j] > \text{arr}[j+1]$, swap and set swapped = true.
4. If nothing was swapped, the array is sorted — stop.

3.1.5 Java implementation

```
1 class Solution {
2
3     public static void bubbleSort(int[] arr) {
4         for (int i = 0; i < arr.length - 1; i++) {
5             boolean swapped = false;
6
7             for (int j = 0; j < arr.length - 1 - i; j++) {
8                 if (arr[j] > arr[j + 1]) {
9                     int temp = arr[j];
10                    arr[j] = arr[j + 1];
11                    arr[j + 1] = temp;
```

```

12             swapped = true;
13         }
14     }
15
16     if (!swapped)
17         break;
18 }
19 }
20 }
```

Listing 3.1: Bubble Sort with early exit

3.1.6 Line by line

<code>i < arr.length - 1</code>	$n - 1$ passes suffice: once $n - 1$ elements are in their final places, the last one has nowhere else to be.
<code>boolean swapped = false</code>	Declared <i>inside</i> the outer loop — it must be reset every pass. Declaring it outside is a classic bug that breaks after the first pass.
<code>j < arr.length - 1 - i</code>	The <code>-1</code> keeps <code>arr[j+1]</code> in bounds; the <code>-i</code> skips the already-finalised suffix. Drop the <code>-i</code> and the code is still <i>correct</i> , just slower. Drop the <code>-1</code> and you get <code>ArrayIndexOutOfBoundsException</code> .
<code>if (arr[j] > arr[j+1])</code> the three <code>temp</code> lines	Strictly greater. Using <code>>=</code> would swap equal elements and destroy stability for no benefit. The standard swap. A <code>temp</code> variable is required because the first assignment would otherwise overwrite the value you still need.
<code>if (!swapped) break</code>	The adaptivity. If a whole pass finds nothing out of order, no inversions remain, so the array is sorted.

3.1.7 Complexity

Comparisons: $\sum_{i=0}^{n-2} (n - 1 - i) = \frac{n(n-1)}{2} = O(n^2)$ worst and average; $O(n)$ best (one pass, then break).
Swaps: $O(n^2)$ worst, 0 best. Space $O(1)$. Stable, in-place, adaptive.

Common Mistake

Declaring `swapped` outside the outer loop. It stays `true` forever after the first swap, so the early exit never fires — the code is correct but silently loses its $O(n)$ best case.

Using `j <= arr.length - 1 - i`. On the first pass this reads `arr[arr.length]` $\Rightarrow$ out of bounds.

Comparing `arr[j] > arr[j-1]` instead of `arr[j+1]` — reads before the start of the array on `j = 0`.

3.1.8 When to use it

Essentially never in production — but it is the cheapest way to *explain* sorting, and interviewers use it to check that you know about the early-exit flag and stability. Its one honest use is detecting whether an array is already sorted in a single pass.

3.2 Selection Sort

3.2.1 Intuition

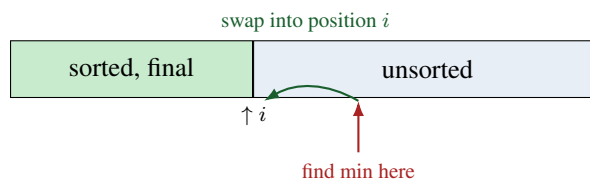
Intuition

Think of dealing yourself a hand of cards laid face up. Look through all the cards you have not yet placed, find the smallest, and put it at the front of your sorted pile. Repeat. You never re-examine the sorted pile, and you make exactly one placement per round — so the number of *moves* is tiny, even though the number of *looks* is large.

3.2.2 Invariant

Invariant

After iteration i , indices $0..i$ hold the $i + 1$ smallest elements of the array, in sorted order and in their final positions.



3.2.3 Example: [5, 3, 8, 1, 2]

i	Array before	Minimum of <code>arr[i..]</code>	After swap
0	[5, 3, 8, 1, 2]	1 at index 3	[1, 3, 8, 5, 2]
1	[1 3, 8, 5, 2]	2 at index 4	[1, 2 8, 5, 3]
2	[1, 2 8, 5, 3]	3 at index 4	[1, 2, 3 5, 8]
3	[1, 2, 3 5, 8]	5 at index 3	[1, 2, 3, 5 8] (self-swap)

Note iteration 3: the minimum is already in place, and we still swap it with itself. The scan happened anyway — that is precisely why Selection Sort cannot adapt.

3.2.4 Java implementation

```

1  class Solution {
2
3      public static void selectionSort(int[] arr) {
4          for (int i = 0; i < arr.length - 1; i++) {
5              int minIdx = i;
6
7              for (int j = i + 1; j < arr.length; j++) {
8                  if (arr[j] < arr[minIdx]) {
9                      minIdx = j;
10                 }
11             }
12
13             swap(arr, i, minIdx);
14         }
15     }
16
17     private static void swap(int[] arr, int i, int j) {
18         int temp = arr[i];
19         arr[i] = arr[j];
20         arr[j] = temp;

```

```

21     }
22 }
```

Listing 3.2: Selection Sort

3.2.5 Line by line

<code>int minIdx = i</code>	We track an <i>index</i> , not a value, because we need to know where to swap from.
<code>j = i + 1</code>	Start after the candidate; comparing <code>arr[i]</code> with itself is pointless.
<code>arr[j] < arr[minIdx]</code>	Strictly less. Using <code><=</code> would move <code>minIdx</code> to the <i>last</i> equal element, making the sort even less stable, for no gain.
<code>swap(arr, i, minIdx)</code>	Outside the inner loop — exactly one swap per outer iteration. Swapping inside the inner loop turns this into a (slower, different) algorithm.

3.2.6 Why it stays $O(n^2)$ even on sorted input

The inner loop is `for (int j = i + 1; j < arr.length; j++)` — no condition inside can stop it early. To be *certain* an element is the minimum of a range you must inspect every element of that range; there is no shortcut, because a single unexamined element could be smaller. So the comparison count is $\sum_{i=0}^{n-2} (n-1-i) = \frac{n(n-1)}{2}$ on *every* input.

Key Insight

Bubble Sort can detect sortedness because a pass that swaps nothing *proves* no inversions remain — the information falls out of the comparisons it was making anyway. Selection Sort’s comparisons only ever ask “is this smaller than my current best?”, which tells it nothing about the order of the elements among themselves. **Adaptivity comes from comparisons whose outcome carries global information.**

3.2.7 Why the number of swaps is small

Exactly $n - 1$ swaps, i.e. $\Theta(n)$ writes — versus up to $\frac{n(n-1)}{2}$ writes for Bubble Sort. If reads are cheap and writes are expensive (EEPROM, flash, network-backed storage), Selection Sort is genuinely the right choice. It is the only context in which it beats Insertion Sort.

3.2.8 Complexity

Time $O(n^2)$ best/average/worst. Swaps $\Theta(n)$. Space $O(1)$. Unstable (see the `[2a, 2b, 1]` example in Chapter 2), in-place, non-adaptive.

Common Mistake

Swapping inside the inner loop (`if (arr[j] < arr[i]) swap(arr, i, j);`). This does sort, but it performs $O(n^2)$ swaps and loses the algorithm’s only advantage.

int min = arr[i] instead of minIdx — you then know the value but not where to swap from.

Outer loop to `i < arr.length` — harmless but wasteful: the final iteration scans an empty range and swaps an element with itself.

3.2.9 When to use it

When writes are the bottleneck, or when you need the k smallest elements for very small k (stop after k iterations — $O(kn)$, competitive with a heap for tiny k).

3.3 Insertion Sort

3.3.1 Intuition

Intuition

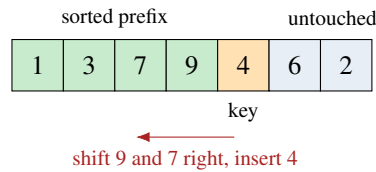
This is how everyone actually sorts a hand of playing cards. You hold a sorted fan in your left hand. You pick up the next card and slide it leftwards past every card that is bigger, until it meets one that is smaller — then you drop it in. Crucially, you never touch the cards to the right of the one you picked up, and you stop sliding the moment you find the right spot. That early stop is the whole reason Insertion Sort is fast on nearly-sorted data.

3.3.2 Invariant

Invariant

At the start of iteration i , the subarray `arr[0..i-1]` is sorted — but *not necessarily in final position*, because later elements may still be inserted into it.

$$\left[\underbrace{\text{sorted prefix}}_{0..i-1} \mid \underbrace{\text{key}}_i \mid \underbrace{\text{untouched}}_{i+1..n-1} \right]$$



3.3.3 Example: [5, 3, 8, 1, 2]

i	key	Shifting	Result
1	3	$5 > 3 \Rightarrow$ shift 5 right	[3, 5 8, 1, 2]
2	8	$5 < 8 \Rightarrow$ no shift	[3, 5, 8 1, 2]
3	1	8,5,3 all $> 1 \Rightarrow$ 3 shifts	[1, 3, 5, 8 2]
4	2	8,5,3 > 2 ; 1 $< 2 \Rightarrow$ 3 shifts, stop	[1, 2, 3, 5, 8]

Detailed trace of $i = 3$, `key` = 1, starting from [3, 5, 8, 1, 2]:

3	5	8	1	2	<code>key</code> = <code>arr</code> [3] = 1, <code>j</code> = 2
3	5	8	8	2	<code>arr</code> [2]=8 > 1: shift, <code>j</code> = 1
3	5	5	8	2	<code>arr</code> [1]=5 > 1: shift, <code>j</code> = 0
3	3	5	8	2	<code>arr</code> [0]=3 > 1: shift, <code>j</code> = -1
1	3	5	8	2	<code>j</code> < 0: insert <code>key</code> at <code>arr</code> [0]

Notice that the value 1 lives only in the variable `key` during the shifting — the array temporarily holds a duplicate. That is fine, because `key` is our safe copy.

3.3.4 Swapping vs shifting

Two ways to move the key into place:

Repeated swapping	Shifting then inserting
<pre> for (int i = 1; i < n; i++) for (int j = i; j > 0 && arr[j-1] > arr[j]]; j--) swap(arr, j, j-1); </pre>	<pre> int key = arr[i]; int j = i - 1; while (j ≥ 0 && arr[j] > key) { arr[j+1] = arr[j]; j--; } arr[j+1] = key; </pre>
3 array writes per position moved	1 array write per position moved
The key is re-read and re-written every step	The key sits in a register the whole time
Simpler to see, slower to run	Standard implementation

Key Insight

Both do the same number of *comparisons*; they differ in *writes*. A swap is three assignments; a shift is one. Moving the key k positions costs $3k$ writes by swapping versus $k + 1$ by shifting — a constant-factor win of roughly $3\times$, which is why shifting is the standard form. Same asymptotics, meaningfully different constant.

3.3.5 Java implementation

```

1  class Solution {
2
3      public static void insertionSort(int[] arr) {
4          for (int i = 1; i < arr.length; i++) {
5              int key = arr[i];
6              int j = i - 1;
7
8              while (j ≥ 0 && arr[j] > key) {
9                  arr[j + 1] = arr[j];
10                 j--;
11             }
12
13             arr[j + 1] = key;
14         }
15     }
16 }

```

Listing 3.3: Insertion Sort — shift and insert

3.3.6 Line by line

<code>i = 1</code>	A single element is already a sorted prefix, so there is nothing to do for <code>i = 0</code> .
<code>int key = arr[i]</code>	Save the value <i>before</i> shifting overwrites its slot.
<code>int j = i - 1</code>	Start comparing at the last element of the sorted prefix and walk left.
<code>j >= 0 && arr[j] > key</code>	Two exit conditions: we ran off the left end (key is the new minimum), or we found an element $\leq$ key (the correct spot). Java's <code>&&</code> short-circuits, so <code>arr[-1]</code> is never evaluated — swapping the two conditions crashes.
<code>arr[j + 1] = arr[j]</code>	The shift: copy the bigger element one slot right, into the hole.
<code>arr[j + 1] = key</code>	After the loop, <code>j</code> points one slot <i>left</i> of the hole, so the hole is at <code>j+1</code> . This is true in both exit cases, including <code>j == -1 $\Rightarrow$ insert at index 0</code> .

3.3.7 Why the best case is $O(n)$

On already-sorted input, `arr[j] > key` is false at the very first test for every `i`, so the `while` body never runs. The work is $n - 1$ comparisons and $n - 1$ redundant self-assignments `arr[i] = key`: $\Theta(n)$ total.

More precisely, the number of times the `while` body executes for a given `i` is exactly the number of elements in `arr[0..i-1]` that are greater than `arr[i]` — that is, the number of inversions involving position `i`. Summing over all `i`:

Insertion Sort's running time = $\Theta(n + \text{number of inversions})$

This identity is the bridge to Chapter 4, and it is the single most useful fact about elementary sorting.

3.3.8 Complexity

Time $O(n)$ best, $O(n^2)$ average and worst (average inversions $\approx n^2/4$). Space $O(1)$. Stable (`arr[j] > key` stops at an equal element, so equals never cross), in-place, strongly adaptive.

Common Mistake

Writing `arr[j] >= key`. Now the key slides past equal elements, destroying stability, and on an array of all-equal values it degrades from $O(n)$ to $O(n^2)$.

Condition order `arr[j] > key && j >= 0`. On the first iteration with `j` reaching -1 this evaluates `arr[-1]` $\Rightarrow$ `ArrayIndexOutOfBoundsException`. The bounds check must come first.

`arr[j] = key` after the loop instead of `arr[j+1] = key` — off by one; overwrites the wrong slot and loses an element.

Starting the outer loop at `i = 0` — harmless, but it does a wasted iteration.

3.3.9 When to use it

- Small arrays ($n \lesssim 16$). Real library sorts (including Java's dual-pivot QuickSort) switch to Insertion Sort for small subarrays, because its constant factor beats the recursion overhead.
- Nearly-sorted data — $O(n + \text{inversions})$ is hard to beat.
- Online / streaming input: it can insert a new element into an already-sorted array in $O(n)$ without re-sorting.
- Linked lists: shifting is cheap, and **LeetCode 147 — Insertion Sort List** is exactly this.

Interview Tip

If asked “which $O(n^2)$ sort would you actually use?”, the answer is Insertion Sort, and the reason is adaptivity plus a small constant factor. If asked “why do industrial sorts use Insertion Sort inside

QuickSort?”, the answer is that below a threshold the $O(n^2)$ term is smaller than the recursion and partitioning overhead.

Practice

Before moving on, without looking back:

1. State what is guaranteed after Bubble pass i , after Selection iteration i , and at the start of Insertion iteration i .
2. Explain why Bubble can exit early but Selection cannot.
3. Trace Insertion Sort on $[4, 3, 2, 1]$ and count the shifts. (You should get $3 + 2 + 1 = 6$ — the maximum, and exactly the inversion count.)

Chapter 4

Inversions — What Sorting Actually Removes

4.1 Definition

Key Insight

For indices $i < j$, the pair (i, j) is an **inversion** if

$$\text{arr}[i] > \text{arr}[j].$$

In plain English: *a pair of elements that appear in the wrong relative order.*

Note carefully that an inversion is a pair of *positions*, not a pair of values, and that the condition involves *both* the index order ($i < j$) and the value order ($\text{arr}[i] > \text{arr}[j]$). The pair must be wrong in exactly one of the two senses.

An array is sorted *if and only if* it has zero inversions. That is not a coincidence or a cute observation — it is a complete characterisation. It means:

Sorting is the process of removing inversions. Nothing more.

Every algorithm in this booklet is a different strategy for destroying inversions: Bubble destroys them one adjacent pair at a time; Insertion destroys all inversions involving one element per iteration; Merge Sort destroys entire blocks of them at once; partitioning destroys all inversions that cross the pivot.

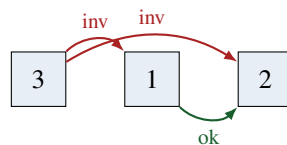
4.2 Worked examples

[3, 1, 2]

Check all $\binom{3}{2} = 3$ pairs:

Pair (i, j)	Values	$\text{arr}[i] > \text{arr}[j]$?	
(0, 1)	3, 1	yes	inversion
(0, 2)	3, 2	yes	inversion
(1, 2)	1, 2	no	fine

Inversion count = 2.



[1, 2, 3, 4]

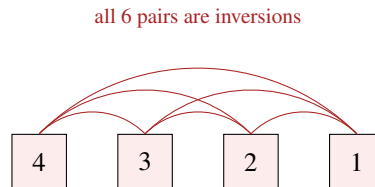
Every pair is in order: **0 inversions**. Already sorted.

[2, 1, 3]

Only (0, 1) is wrong: **1 inversion**. One adjacent swap fixes it.

[4, 3, 2, 1]

Every one of the $\binom{4}{2} = 6$ pairs is an inversion: **6 inversions** — the maximum for $n = 4$.



4.3 The maximum number of inversions

An inversion is a choice of two positions, and for a strictly decreasing array *every* choice is an inversion. The number of ways to choose 2 positions from n is

$$\binom{n}{2} = \frac{n(n-1)}{2}.$$

So a reverse-sorted array of distinct elements has exactly $\frac{n(n-1)}{2}$ inversions, and no array can have more. For $n = 5$ that is 10; for $n = 1000$ it is 499,500.

Array shape	Inversions	Insertion Sort cost	Called
sorted	0	$\Theta(n)$	best case
one element out of place	$O(n)$	$\Theta(n)$	nearly sorted
random permutation	$\approx \frac{n^2}{4}$ (expected)	$\Theta(n^2)$	average case
reverse sorted	$\frac{n(n-1)}{2}$	$\Theta(n^2)$	worst case

Why the average is $n^2/4$. For a uniformly random permutation, each of the $\binom{n}{2}$ pairs is inverted with probability exactly $\frac{1}{2}$ (by symmetry: the two orderings of any pair are equally likely). By linearity of expectation the expected inversion count is $\frac{1}{2}\binom{n}{2} = \frac{n(n-1)}{4} \approx \frac{n^2}{4}$.

4.4 Why inversions matter

4.4.1 Insertion Sort: a machine for removing inversions

Recall the inner loop:

```
while (j ≥ 0 && arr[j] > key) {
    arr[j + 1] = arr[j];    // shift one element right
    j--;
}
```

The body executes once for each element of the sorted prefix that is greater than the key. But “an element at a smaller index that is greater than `arr[i]`” is precisely the definition of an inversion with right endpoint i . Therefore:

Key Insight

Each shift in Insertion Sort resolves exactly one inversion. Iteration i performs k_i shifts, where k_i is the number of inversions (j, i) with $j < i$. Since every inversion has exactly one right endpoint, $\sum_i k_i$ is the total inversion count, and

$$T_{\text{insertion}}(\text{arr}) = \Theta(n + \text{inv}(\text{arr})).$$

This one formula explains everything we observed in Chapter 3:

Nearly sorted $\Rightarrow$ few inversions $\Rightarrow$ few shifts $\Rightarrow$ Insertion Sort is fast ($\rightarrow O(n)$)

Reverse sorted $\Rightarrow \frac{n(n-1)}{2}$ inversions $\Rightarrow$ maximum shifts $\Rightarrow$ Insertion Sort is slow ($\Theta(n^2)$)

“Adaptive” now has a precise meaning: *the running time is a function of the inversion count, not just of n* . It is not a vague property of the implementation; it falls straight out of the loop structure.

4.4.2 Bubble Sort: one adjacent swap, one inversion

A swap of *adjacent* elements changes the relative order of exactly one pair — the two elements being swapped — and leaves every other pair’s relative order untouched. So each Bubble Sort swap also removes exactly one inversion, and the total number of swaps Bubble Sort performs equals the inversion count.

Key Insight

Since one adjacent swap removes at most one inversion, **any** sorting algorithm restricted to adjacent swaps needs at least $\text{inv}(\text{arr})$ swaps, hence $\Omega(n^2)$ in the worst case. This is a real lower bound, and it tells you why beating $O(n^2)$ *requires* moving elements over long distances — which is exactly what Merge Sort’s merge and QuickSort’s partition do.

4.4.3 Selection Sort: the exception that proves the rule

Selection Sort’s swaps are long-distance, so a single swap can remove many inversions at once (it can also *create* some). That is why its swap count is $\Theta(n)$ regardless of the input — and also why its running time is *not* a function of the inversion count: the cost is in the scanning, not the moving.

4.5 Inversions as a problem-solving tool

Inversion counting is a technique in its own right, well beyond sorting. The pattern to recognise is:

“count pairs (i, j) with $i < j$ satisfying some order-reversing condition”

Instances you will meet:

Classic inversion count	pairs with $i < j$ and $a[i] > a[j]$
LeetCode 493 — Reverse Pairs	pairs with $i < j$ and $a[i] > 2 \cdot a[j]$ — same technique, a different comparison in the counting step
LeetCode 315 — Count of Smaller Numbers After Self	for each i , how many $j > i$ have $a[j] < a[i]$ — inversions bucketed by left endpoint
Minimum adjacent swaps to sort	equals the inversion count exactly (by the argument above)
Measuring “sortedness”	Kendall’s τ rank correlation is a normalised inversion count — used in recommender systems to compare two rankings

The naive way to count inversions is two nested loops, $O(n^2)$. Chapter 7 shows how the merge step of Merge Sort counts them in $O(n \log n)$ almost for free — and that trick is the reason this chapter appears *before* Merge Sort rather than after it.

Practice

1. Count the inversions in $[2, 4, 1, 3, 5]$ by hand. (Answer: 3 — namely $(2, 1)$, $(4, 1)$, $(4, 3)$.)
2. Verify by tracing Insertion Sort on it and counting shifts. You must get the same number.
3. An array of n distinct elements has $\frac{n(n-1)}{2} - 1$ inversions. What does it look like? (Hint: exactly one pair is in the correct order — which pair can it be?)
4. If you swap two *adjacent* elements of an array, how much can the inversion count change? What if you swap two arbitrary elements?

Interview Tip

When a problem says “count pairs”, “how many $j > i$ such that...”, or “minimum adjacent swaps”, say the word *inversions* out loud and reach for merge sort or a Binary Indexed Tree. Interviewers are usually testing whether you can get from $O(n^2)$ to $O(n \log n)$, and naming the pattern is half the answer.

Chapter 5

Merge Sort

5.1 Start with the merge, not the recursion

Merge Sort is almost always taught backwards. The recursion is the easy part; the idea that makes it work is this:

Intuition

If I already have two *sorted* sequences, I can combine them into one sorted sequence in a single linear pass. Look at the front of each; whichever is smaller must be the overall smallest remaining element, so take it and advance that pointer. Repeat. No comparison is ever wasted, and each element is touched once.

Why is the smaller front element necessarily the global minimum? Because each sequence is sorted, so its front is the smallest of *its* remaining elements. The global minimum must be the smaller of those two.

5.1.1 The two-pointer merge

Left [1, 5, 8] Right [2, 3, 7] $\longrightarrow$ [1, 2, 3, 5, 7, 8]

$i \rightarrow \text{left}$	1 5 8	temp = []
$j \rightarrow \text{right}$	2 3 7	temp = [1]
	1 5 8	temp = [1, 2]
	2 3 7	temp = [1, 2, 3]
	1 5 8	temp = [1, 2, 3, 5]
	2 3 7	temp = [1, 2, 3, 5, 7]
	1 5 8	temp = [1, 2, 3, 5, 7, 8]
	2 3 7	
	1 5 8	
	2 3 7	
	1 5 8	
	2 3 7	

Read it as: compare `left[i]` with `right[j]`, append the smaller to `temp`, advance that pointer. When one side runs out, copy the rest of the other side verbatim — it is already sorted and every element is larger than everything emitted so far.

Invariant

- `temp[0..k-1]` is sorted;
- every element of `temp[0..k-1]` is $\leq$ every remaining element of both inputs;
- $k == (i - 1) + (j - m - 1)$: the number emitted equals the number consumed.

Merging two sorted ranges of total length n costs exactly n writes and at most $n - 1$ comparisons: $\Theta(n)$.

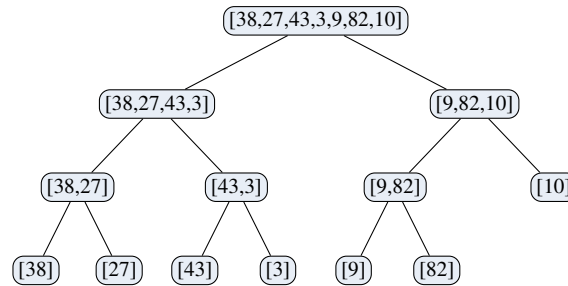
5.2 From merge to Merge Sort: divide and conquer

We now have a linear-time combiner but no sorted halves. The leap:

Key Insight

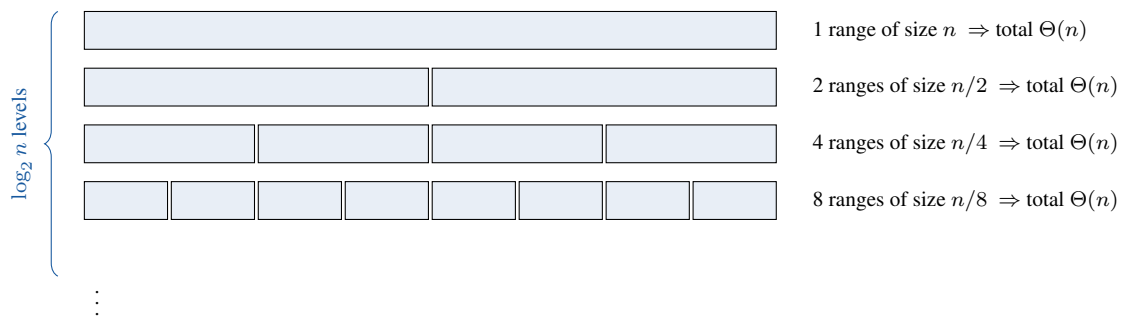
Make the halves sorted — by the same method. A single element is trivially sorted, so recursion bottoms out immediately.

Divide the range in half → **Solve** each half recursively → **Combine** with merge.



5.2.1 Why $O(n \log n)$ — the intuitive derivation

Count the work *level by level* rather than call by call.



At every level the ranges are disjoint and together cover all n elements, so all the merging at that level costs $\Theta(n)$ — regardless of how many ranges there are. Halving repeatedly reaches size 1 after $\log_2 n$ steps. Hence

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n).$$

Key Insight

The two factors have distinct sources and you should be able to name them separately: n is the cost of merging one level; $\log n$ is the number of times you can halve n before reaching 1. Nothing about the data affects either, which is why Merge Sort is $\Theta(n \log n)$ in the best case too — and why it is not adaptive.

5.3 Top-down (recursive) Merge Sort

```

1 class Solution {
2
3     public static void mergeSort(int[] arr, int l, int r) {
4         if (l ≥ r)
5             return;                                // 0 or 1 element: already sorted
  
```

```

6
7     int m = 1 + (r - 1) / 2;           // midpoint, overflow-safe
8
9     mergeSort(arr, l, m);             // sort left half [l, m]
10    mergeSort(arr, m + 1, r);         // sort right half [m+1, r]
11
12    merge(arr, l, m, r);              // combine the two sorted halves
13 }
14
15 private static void merge(int[] arr, int l, int m, int r) {
16     int[] temp = new int[r - l + 1];
17
18     int i = l;           // cursor in the left half
19     int j = m + 1;       // cursor in the right half
20     int k = 0;           // cursor in temp
21
22     while (i ≤ m && j ≤ r) {
23         if (arr[i] ≤ arr[j])           // tie: take LEFT, keeps it STABLE
24             temp[k++] = arr[i++];
25         else
26             temp[k++] = arr[j++];
27     }
28
29     while (i ≤ m)                   // drain the left half
30         temp[k++] = arr[i++];
31
32     while (j ≤ r)                   // drain the right half
33         temp[k++] = arr[j++];
34
35     for (int x = 0; x < temp.length; x++)
36         arr[l + x] = temp[x];       // copy back, offset by l
37 }
38 }

```

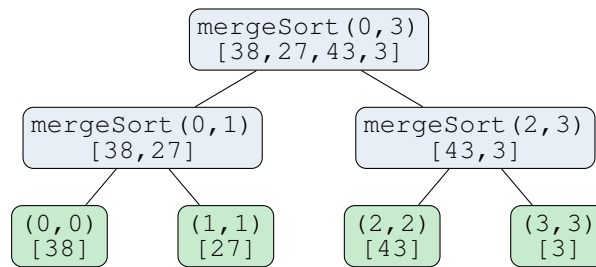
Listing 5.1: JTop-down Merge Sort; ranges are inclusive [l..r]

Entry point: `mergeSort(arr, 0, arr.length - 1);`

5.3.1 Line by line

<code>if (l >= r) return;</code>	The base case. <code>l == r</code> is a single element; <code>l > r</code> is empty (reachable only from a bad call). Writing <code>if (l == r)</code> instead is a real bug: an empty range would recurse forever.
<code>int m = 1 + (r - 1) / 2;</code>	Equivalent to $\lfloor (l + r) / 2 \rfloor$ but immune to integer overflow when $l + r > 2^{31} - 1$. Java's own <code>Arrays.binarySearch</code> had exactly this bug for nine years.
<code>mergeSort(arr, l, m)</code>	The left half <i>includes</i> <code>m</code> .
<code>mergeSort(arr, m + 1, r)</code>	The right half starts <i>after</i> <code>m</code> . The two halves must partition the range with no overlap and no gap — <code>[l, m]</code> and <code>[m+1, r]</code> do exactly that.
<code>new int[r - l + 1]</code>	Size of an inclusive range. Off-by-one here truncates the last element.
<code>i = l, j = m + 1</code>	Cursors into the two halves <i>of the original array</i> — we never physically split it.
<code>arr[i] <= arr[j]</code>	On a tie, take from the left . That is what makes Merge Sort stable. <code><</code> would take from the right on ties and break stability.
the two drain loops	Exactly one of them runs. Forgetting them leaves zeros in <code>temp</code> .
<code>arr[l + x] = temp[x]</code>	<code>temp</code> is 0-based but the range starts at <code>l</code> , so the offset is required. Writing <code>arr[x]</code> corrupts the array.

5.3.2 Full recursion trace: [38, 27, 43, 3]



Execution order (depth-first, left before right), with the array state after each merge:

#	Call	Action	temp	Array after
1	(0, 3)	$m = 1$; recurse left		[38, 27, 43, 3]
2	(0, 1)	$m = 0$; recurse left		
3	(0, 0)	base, return		
4	(1, 1)	base, return		
5	merge(0, 0, 1)	$27 < 38$	[27, 38]	[27, 38, 43, 3]
6	(2, 3)	$m = 2$; two bases		
7	merge(2, 2, 3)	$3 < 43$	[3, 43]	[27, 38, 3, 43]
8	merge(0, 1, 3)	merge [27, 38] with [3, 43]	[3, 27, 38, 43]	[3, 27, 38, 43]

Step 8 in detail: i at 27, j at 3 $\Rightarrow$ take 3. Then 27 vs 43 $\Rightarrow$ 27. Then 38 vs 43 $\Rightarrow$ 38. Left exhausted; drain right $\Rightarrow$ 43.

Key Insight

Notice the shape of the execution: *all* the dividing happens on the way *down*, and *all* the real work (merging) happens on the way *up*. This is the opposite of QuickSort, where all the work happens on the way down (partitioning) and the return trip does nothing. Hold on to that contrast — it is the cleanest way to remember the difference between the two algorithms.

5.3.3 Complexity

- **Time:** $\Theta(n \log n)$ best, average and worst. The recursion depth is $\lceil \log_2 n \rceil$ and each level merges $\Theta(n)$ elements.
- **Space:** $O(n)$ for `temp` (allocated per merge here; a single shared buffer is the usual optimisation), plus $O(\log n)$ recursion stack.
- **Stable:** yes, thanks to $\leq$.
- **In-place:** no.
- **Adaptive:** no — although a one-line check `if (arr[m] <= arr[m+1]) return;` before merging makes it $O(n)$ on sorted input, which is the core idea behind TimSort.

Common Mistake

`int m = (1 + r) / 2;` — overflows for large indices. Use `1 + (r - 1) / 2`.

`mergeSort(arr, m, r)` instead of `m + 1` — the element at `m` belongs to both halves, so the range never shrinks $\Rightarrow$ infinite recursion.

Forgetting the drain loops — the tail of the longer half is silently replaced by zeros.

`arr[x] = temp[x]` in the copy-back — forgets the `+ 1` offset and writes the merged block to the wrong place.

Using `<` instead of `<=` in the comparison — still sorts correctly for `int[]`, but is unstable, which

silently breaks multi-key sorting of objects.

5.3.4 When to use it

- You need a **guaranteed** $O(n \log n)$ — no adversarial input can degrade it.
- You need **stability** (sorting objects by a secondary key).
- You are sorting a **linked list**: merging needs no random access and no extra array, giving $O(1)$ auxiliary space (**LeetCode 148 — Sort List**).
- You are sorting data that does not fit in memory (external merge sort).
- You need to **count inversions** or answer similar cross-pair queries (Chapter 7) — QuickSort cannot do this.

Interview Tip

“Merge Sort or QuickSort?” has a crisp answer: QuickSort for raw in-memory speed on primitives (cache-friendly, in-place); Merge Sort when you need stability, a worst-case guarantee, linked lists, external data, or inversion counting. Java itself makes exactly this choice — `Arrays.sort(int[])` uses dual-pivot QuickSort, while `Arrays.sort(Object[])` uses TimSort, a stable merge sort.

Chapter 6

Bottom-Up (Iterative) Merge Sort

6.1 The idea: stop dividing, start doubling

Look again at the recursion tree from Chapter 5. All the dividing on the way down accomplishes exactly one thing: it decides *which pairs of adjacent blocks get merged, and in what order of block size*. But we can write those block sizes down directly:

Intuition

The recursion merges blocks of size 1 into blocks of size 2, then size 2 into size 4, then 4 into 8, and so on. If I know that in advance, I do not need the recursion at all — I just loop over the block width, doubling it each round, and merge every adjacent pair of blocks of that width.

Key Insight

Top-down discovers the block sizes by dividing, then merges on the way back up.

Bottom-up states the block sizes up front and merges immediately.

Same merges, same total work, same $\Theta(n \log n)$ — but no recursion, hence no call stack. The outer `for` loop over `width` is the recursion depth, made visible.

Invariant

Before the pass with a given `width`, every block `arr[k*width .. min(k*width + width - 1, n-1)]` is individually sorted. After the pass, every block of size `2*width` is sorted. When `width` $\geq n$, the single remaining block is the whole array.

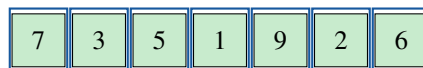
6.2 Full trace on an odd-sized array

Odd sizes are where bottom-up merge sort actually gets interesting, because blocks stop lining up. Take

`[7, 3, 5, 1, 9, 2, 6]` ($n = 7$)

width = 1

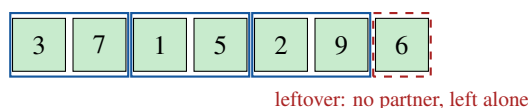
Every single element is trivially a sorted block:



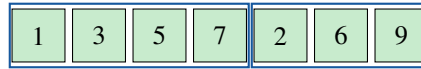
We merge the pairs (0, 1), (2, 3), (4, 5) and then reach index 6 — a block with no partner.

width = 2

(result of merging pairs of 1-blocks)



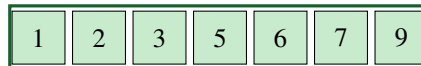
`[3, 7]` `[1, 5]` `[2, 9]` `[6]`

width = 4(merge $[3, 7] + [1, 5]$, and $[2, 9] + [6]$)

$[1, 3, 5, 7]$ $[2, 6, 9]$ — note the second block has length 3, not 4. That is fine: the merge routine takes explicit `left`, `mid`, `right` indices and does not care that the block is short.

width = 8

(merge the two remaining blocks)

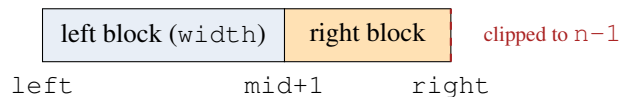


Sorted. Since $8 \geq 7$, the outer loop now stops.

6.3 How incomplete final blocks are handled

This is the part that trips people up, and it is handled by two `Math.min` calls plus one guard.

For a pass with block size `width` starting at index `left`:



```
int mid = Math.min(left + width - 1, n - 1);
int right = Math.min(left + 2 * width - 1, n - 1);
if (mid < right)
    merge(arr, left, mid, right);
```

<code>Math.min(..., n - 1)</code> <code>on mid</code>	The left block itself may run off the end (e.g. <code>left = 6, width = 4, n = 7</code>). Clipping keeps <code>mid</code> inside the array.
<code>Math.min(..., n - 1)</code> <code>on right</code>	The right block is the one that is usually short. Clipping means we merge only the elements that actually exist.
<code>if (mid < right)</code>	The guard for a <i>missing</i> right block. If clipping made <code>mid == right</code> (or <code>mid > right</code>), there is no second block to merge with — the leftover is already sorted from the previous pass, so we simply leave it. This is exactly the <code>[6]</code> case at width 2 above.

Key Insight

A leftover block is never *lost*: it sits untouched until some later, wider pass finds it a partner. At width 2 the block `[6]` had no partner; at width 4 it was absorbed into $[2, 9] + [6] = [2, 6, 9]$; at width 8 that merged with everything else. Because the width doubles, any leftover is picked up within $O(\log n)$ passes.

6.4 Java implementation

```

1  class Solution {
2
3      public static void mergeSortIterative(int[] arr) {
4          int n = arr.length;
5
6          for (int width = 1; width < n; width *= 2) {
7
8              for (int left = 0; left < n; left += 2 * width) {
9
10                 int mid    = Math.min(left + width - 1,      n - 1);
11                 int right = Math.min(left + 2 * width - 1, n - 1);
12
13                 if (mid < right)
14                     merge(arr, left, mid, right);
15             }
16         }
17     }
18
19     // identical to the merge used by top-down merge sort
20     private static void merge(int[] arr, int l, int m, int r) {
21         int[] temp = new int[r - l + 1];
22         int i = l, j = m + 1, k = 0;
23
24         while (i ≤ m && j ≤ r) {
25             if (arr[i] ≤ arr[j])
26                 temp[k++] = arr[i++];
27             else
28                 temp[k++] = arr[j++];
29         }
30         while (i ≤ m) temp[k++] = arr[i++];
31         while (j ≤ r) temp[k++] = arr[j++];
32
33         for (int x = 0; x < temp.length; x++)
34             arr[l + x] = temp[x];
35     }
36 }

```

Listing 6.1: Bottom-up Merge Sort

Key Insight

The merge function is **byte-for-byte the same** as in top-down Merge Sort. Only the driver changed. That is the whole lesson of this chapter: *merge* is the mechanism; recursion and the width loop are two interchangeable ways of scheduling it.

6.5 Why `width *= 2`?

Because each pass turns sorted blocks of size `width` into sorted blocks of size `2 * width`. The next pass must operate on the blocks that now exist.

	<code>width *= 2</code>	<code>width++</code>
Blocks after pass at width w	sorted blocks of size $2w$	sorted blocks of size $2w$
Next pass assumes blocks of size	$2w$ ✓ correct	$w + 1$ ✗ wrong
Number of passes	$\log_2 n$	$n/2$
Total time	$\Theta(n \log n)$	$\Theta(n^2)$ and incorrect

With `width++` the pass at width 2 would try to merge `arr[0..1]` with `arr[2..3]` — but after the width-1 pass followed by a width-2 pass, the array is already in blocks of 4, so the assumed block boundaries no longer match reality and the output is not sorted. Doubling is not an optimisation; it is a **correctness** requirement, because it is the only progression that matches what the previous pass produced.

6.6 Why the condition is `width < n`, not `width < n - 1`

The loop must run once for every block size strictly smaller than the array. Consider $n = 2$:

Condition	<code>width = 1: 1 < n?</code>	Outcome
<code>width < n</code> ($n = 2$)	$1 < 2$ ✓	merges <code>[0..0]</code> with <code>[1..1]</code> — sorted
<code>width < n - 1</code> ($n = 2$)	$1 < 1$ ✗	loop never runs — array returned unsorted

The general statement: with n elements, the last pass needed has $\text{width} = 2^{\lceil \log_2 n \rceil - 1}$, which is the largest power of two strictly less than n . `width < n` admits exactly those widths. `width < n - 1` loses the final pass whenever n is a power of two plus one, and breaks $n = 2$ outright.

Common Mistake

`width <= n` — harmless but performs one useless extra pass in which the guard `mid < right` always fails.

`left += width` instead of `left += 2 * width` — the inner loop then steps by one block instead of two, so blocks get merged twice and overlapping merges corrupt the array.

Omitting `Math.min` — `ArrayIndexOutOfBoundsException` on any n that is not a power of two.

Omitting `if (mid < right)` — calls `merge` with `m == r`, which merges a block with an empty range: usually harmless but wasteful, and with a buggy merge it can silently drop elements.

`int mid = left + width` (forgetting `- 1`) — `mid` must be the *last* index of the left block, since our merge uses inclusive ranges.

6.7 Top-down vs bottom-up

	Top-down (recursive)	Bottom-up (iterative)
Control	recursion	two nested loops
Merge order	depth-first: finishes the whole left subtree first	breadth-first: one complete width at a time
Stack space	$O(\log n)$	$O(1)$
Block boundaries	computed by halving	computed by doubling
Odd sizes	handled automatically by the midpoint	handled by <code>Math.min</code> + guard
Readability	clearer	needs the width picture
Linked lists	needs the middle (slow/fast pointers)	natural fit, avoids finding the middle

Interview Tip

LeetCode 148 — Sort List asks for $O(n \log n)$ time and $O(1)$ space on a linked list. The recursive solution uses $O(\log n)$ stack, so strictly it fails the space requirement — the bottom-up version is the intended answer. Being able to explain *why* bottom-up removes the stack is exactly what that problem tests.

Practice

1. Trace bottom-up merge sort on $[4, 1, 3, 9, 7]$ ($n = 5$). At which width does the leftover $[7]$ first get a partner?
2. For $n = 16$ and $n = 17$, how many passes does the outer loop make?
3. Modify the code to count how many times `merge` is actually called for $n = 7$. (You should get $3 + 2 + 1 = 6$.)

Chapter 7

Counting Inversions with Merge Sort

7.1 The problem

Given an array, count the pairs (i, j) with $i < j$ and $\text{arr}[i] > \text{arr}[j]$.

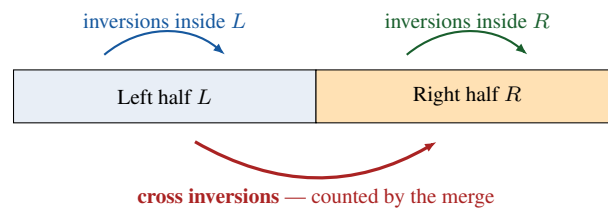
The brute-force solution is two nested loops, $O(n^2)$. We want $O(n \log n)$. The route there is a classic divide-and-conquer decomposition:

Key Insight

Split the array into a left half L and a right half R . Every inversion (i, j) falls into exactly one of three groups:

1. both i and j in L — counted by the recursive call on L ;
2. both i and j in R — counted by the recursive call on R ;
3. i in L and j in R — **cross inversions**, counted during the merge.

These three groups are disjoint and exhaustive, so the total is their sum. Since the merge is what looks at L and R together, the merge is the only place cross inversions can be counted — and it turns out it can count them all in $O(n)$.



7.2 Counting cross inversions during the merge

Here is the crucial observation. Suppose both halves are already sorted, and we are merging

$$L = [2, 5, 8] \quad R = [1, 3, 7]$$

with cursors i into L and j into R .

Key Insight

When the merge takes an element from the *right* — that is, when $\text{right}[j] < \text{left}[i]$ — then $\text{right}[j]$ is smaller than $\text{left}[i]$ *and*, because L is sorted, it is smaller than every remaining element $\text{left}[i], \text{left}[i+1], \dots, \text{left}[m]$ too. Every one of those elements sits at a smaller index than $\text{right}[j]$ in the original array. So this single comparison reveals

$$(m - i + 1) \text{ inversions at once.}$$

That is why the count can be accumulated in linear time per merge instead of quadratic: one comparison discovers a whole block of inversions rather than one.

The worked example

$L = [2, 5, 8], R = [1, 3, 7]$, with $m - i + 1$ counting the elements still unused in L .

Compare	Take	Remaining in L	Inversions added
2 vs 1	1 (right)	{2, 5, 8}	3
2 vs 3	2 (left)	{5, 8}	0
5 vs 3	3 (right)	{5, 8}	2
5 vs 7	5 (left)	{8}	0
8 vs 7	7 (right)	{8}	1
—	8 (drain left)	{}	0

Total cross inversions = $3 + 2 + 1 = 6$.



taking 1 resolves 3 inversions at once

Verify by hand

The unsorted original might have been $[2, 5, 8, 1, 3, 7]$. Its cross inversions are $(2, 1), (5, 1), (8, 1), (5, 3), (8, 3), (8, 7)$ — six of them, matching. And notice: we never enumerated the pairs. We inferred their count from the sortedness of L .

7.3 Why the total is $O(n \log n)$

The recursion is identical to Merge Sort's:

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n).$$

The counting adds only a single addition inside the existing merge loop, so it does not change the asymptotics. The sorting is not a side effect we tolerate — it is the *engine*: the count is only valid because each half is sorted when we merge it.

Key Insight

This is a general and reusable technique, not a trick specific to inversions. Whenever a quantity decomposes as $(left) + (right) + (cross\ pairs)$, and the cross term becomes cheap once both halves are sorted, merge sort will compute it in $O(n \log n)$. That is the entire idea behind **LeetCode 493 — Reverse Pairs** and **LeetCode 315 — Count of Smaller Numbers After Self**.

7.4 Java implementation

```

1  class Solution {
2
3      public static long countInversions(int[] arr) {
4          int[] temp = new int[arr.length];           // reused buffer
5          return sortAndCount(arr, temp, 0, arr.length - 1);
6      }
7
8      private static long sortAndCount(int[] arr, int[] temp, int l, int r) {
9          if (l ≥ r)
10             return 0;                               // 0 or 1 element
11
12         int m = l + (r - l) / 2;
13

```

```

14     long count = 0;
15     count += sortAndCount(arr, temp, l, m);           // inversions inside left
16     count += sortAndCount(arr, temp, m + 1, r);       // inversions inside right
17     count += mergeAndCount(arr, temp, l, m, r);       // cross inversions
18
19     return count;
20 }
21
22 private static long mergeAndCount(int[] arr, int[] temp,
23                                   int l, int m, int r) {
24     int i = l;           // cursor in left half [l, m]
25     int j = m + 1;       // cursor in right half [m+1, r]
26     int k = l;           // cursor in temp
27
28     long count = 0;
29
30     while (i ≤ m && j ≤ r) {
31         if (arr[i] ≤ arr[j]) {
32             temp[k++] = arr[i++];           // no inversion: left is smaller
33         } else {
34             temp[k++] = arr[j++];           // arr[j] < arr[i]
35             count += (m - i + 1);           // ALL remaining left elements
36             // form inversions with arr[j]
37         }
38     }
39
40     while (i ≤ m) temp[k++] = arr[i++];
41     while (j ≤ r) temp[k++] = arr[j++];
42
43     for (int x = l; x ≤ r; x++)
44         arr[x] = temp[x];
45
46     return count;
47 }

```

Listing 7.1: Counting inversions in $O(n \log n)$

7.4.1 Line by line

<code>long return type</code>	The count can reach $\frac{n(n-1)}{2}$. For $n = 10^5$ that is about 5×10^9 , which overflows a 32-bit <code>int</code> . This is a genuine LeetCode failure mode.
<code>int[] temp allocated once</code>	Reusing one buffer avoids $O(n \log n)$ allocations. Note it is indexed by the <i>absolute</i> position $k = 1$, so no $+1$ offset is needed on copy-back.
<code>if (l >= r) return 0;</code>	A range of size ≤ 1 has no inversions.
<code>the three count += lines</code>	The three disjoint groups from the decomposition, in order. Reading them aloud is the proof of correctness.
<code>arr[i] <= arr[j]</code>	Not an inversion: the left element is $\leq$ the right one and comes first. Also keeps the sort stable. Using $<$ here would count pairs of <i>equal</i> elements as inversions, which is wrong by definition ($>$, not $\geq$).
<code>count += (m - i + 1)</code>	The heart of the algorithm. i is the first unused index in the left half and m its last index, so $m - i + 1$ is how many left elements remain — every one of them is $> arr[j]$ and sits at a smaller original index.
<code>placement of count +=</code>	After <code>arr[j++]</code> , but computed from <code>i</code> , which has not moved. Incrementing <code>i</code> instead of <code>j</code> here is the classic bug.
<code>the drain loops</code>	Draining the <i>left</i> half adds nothing: those elements are larger than everything already emitted, but every right-half element has already been accounted for.
<code>arr[x] = temp[x]</code>	Copy back so the parent merge sees sorted halves. Skip this and the counts at higher levels become meaningless.

7.4.2 Dry run on [2, 4, 1, 3, 5]

Expected answer: 3, namely (2, 1), (4, 1), (4, 3).

#	Call	Merge performed	Count
1	<code>(0, 4), m = 2</code>	—	
2	<code>(0, 2), m = 1</code>	—	
3	<code>(0, 1)</code>	<code>merge [2], [4] → [2, 4]</code>	+0
4	<code>(2, 2)</code>	base	0
5	<code>merge(0, 1, 2)</code>	<code>[2, 4] + [1]: take 1, 2 left remain</code> result <code>[1, 2, 4]</code>	+2
6	<code>(3, 4)</code>	<code>merge [3], [5] → [3, 5]</code>	+0
7	<code>merge(0, 2, 4)</code>	<code>[1, 2, 4] + [3, 5]:</code> take 1 (left), 2 (left), then 3 (right) with 1 left remaining (the 4) then 4, then 5 $\Rightarrow [1, 2, 3, 4, 5]$	+1
Total			3 ✓

Step 5 caught (2, 1) and (4, 1); step 7 caught (4, 3). Every inversion counted exactly once, at the unique level where the two elements first end up in different halves.

Common Mistake

Using `int` for the count — overflows for $n \gtrsim 65,000$.

`count += (m - i)` — off by one; forgets that `arr[i]` itself is one of the remaining elements.

`count += (r - j + 1)` — counts remaining *right* elements. Wrong side: inversions are counted from the left half's leftovers.

Adding to the count in the drain loops — double-counts. Everything was already counted in the main loop.

Using $<$ instead of $\leq$ — counts equal-valued pairs as inversions. $[1, 1]$ would report 1 instead of 0.

Forgetting the copy-back — the parent then merges unsorted halves, and the whole counting argument collapses.

7.4.3 Complexity

Time $\Theta(n \log n)$; space $O(n)$ for `temp` plus $O(\log n)$ stack.

Practice

1. Run the algorithm mentally on $[4, 3, 2, 1]$ and check you get 6.
2. **LeetCode 493 — Reverse Pairs** asks for pairs with `arr[i] > 2 * arr[j]`. The merge counting step no longer aligns with the comparison used to merge — so you need a *separate* counting pass over the two sorted halves before merging. Try to write it. Why can't you fold it into the same loop?
3. **LeetCode 315 — Count of Smaller Numbers After Self** wants a per-index answer, not a total. Hint: sort *indices* rather than values so each element can carry its own counter.

Interview Tip

Say the decomposition out loud before writing any code: “inversions inside the left, plus inside the right, plus crossing — and the crossing ones are cheap because both halves are sorted.” That sentence is the solution; the code is bookkeeping.

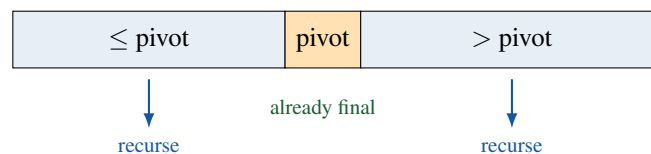
Chapter 8

QuickSort and Partitioning

8.1 The core idea: partition first

Intuition

Pick one element and call it the **pivot**. Now rearrange the array so that everything $\leq$ the pivot is on its left and everything $>$ the pivot is on its right. You have not sorted anything — but you have learned something enormous: no element will ever need to cross the pivot again. The left group and the right group are now completely independent problems, and you can attack each with the same trick.



8.2 QuickSort vs Merge Sort: where the work happens

	Merge Sort	QuickSort
Divide step	trivial: compute the midpoint	expensive: partition, $O(n)$
Combine step	expensive: merge, $O(n)$	trivial: nothing to do
Work happens	on the way <i>up</i>	on the way <i>down</i>
Split sizes	always exactly half	data-dependent; can be $1 : n - 1$
Worst case	$O(n \log n)$ guaranteed	$O(n^2)$
Extra memory	$O(n)$ buffer	$O(1)$ (plus stack)
Stable	yes	no

Key Insight

Merge Sort *divides blindly and combines cleverly*. QuickSort *divides cleverly and combines trivially*. Both are $2T(n/2) + \Theta(n)$; they just put the $\Theta(n)$ in different places. And that is also where their weaknesses come from: Merge Sort's $\Theta(n)$ needs a buffer, while QuickSort's split sizes depend on the pivot and can degenerate.

Since QuickSort's combine step is empty, the recursion is:

```
quickSort(range):
    if range has ≤ 1 element: return
    p = partition(range)           // all the work
    quickSort(left part)
    quickSort(right part)

                                // nothing here
```

Everything now depends on `partition`. There are two standard schemes, and they behave differently in ways that matter.

8.3 Lomuto Partition

8.3.1 Intuition

Intuition

Take the last element as the pivot. Sweep a scanning pointer j from left to right. Maintain a boundary i marking “everything before here is $\leq$ pivot”. Whenever j finds an element that belongs on the small side, swap it into position i and push the boundary right by one. At the end, put the pivot itself at the boundary — the one slot that separates the two groups.

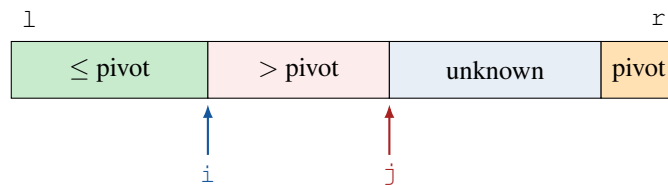
8.3.2 Invariant

Invariant

At every point of the loop, with `pivot = arr[r]`:

$$\left[\underbrace{\leq \text{pivot}}_{l \dots i-1} \mid \underbrace{> \text{pivot}}_{i \dots j-1} \mid \underbrace{\text{unknown}}_{j \dots r-1} \mid \underbrace{\text{pivot}}_r \right]$$

i is where the next small element will go; j is the next element to inspect.



8.3.3 Java implementation

```

1  class Solution {
2
3      public static int lomutoPartition(int[] arr, int l, int r) {
4          int pivot = arr[r];
5          int i = l;
6
7          for (int j = l; j < r; j++) {
8              if (arr[j] ≤ pivot) {
9                  swap(arr, i, j);
10                 i++;
11             }
12         }
13
14         swap(arr, i, r);
15
16         return i;
17     }
18
19     private static void swap(int[] arr, int a, int b) {
20         int temp = arr[a];
21         arr[a] = arr[b];
22         arr[b] = temp;
23     }
24 }

```

Listing 8.1: Lomuto partition; pivot is the last element

8.3.4 Line by line

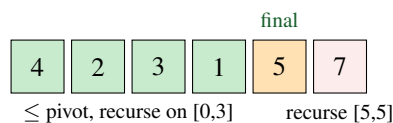
<code>int pivot = arr[r]</code>	Save the <i>value</i> . Position <code>r</code> is excluded from the scan and holds the pivot until the very end.
<code>int i = l</code>	The small-region boundary starts empty.
<code>j < r</code>	Strictly less than <code>r</code> : we must not compare the pivot with itself, or it would be swapped into the small region and the final swap would misplace it.
<code>arr[j] <= pivot</code>	Elements equal to the pivot go <i>left</i> . (This choice is what makes Lomuto degrade to $O(n^2)$ on arrays of all-equal elements — see the mistakes box.)
<code>swap(arr, i, j); i++;</code>	Move the small element to the boundary and extend the small region. When <code>i == j</code> the swap is a no-op, which is the common case at the start.
<code>swap(arr, i, r)</code>	Place the pivot at the boundary. Everything left of <code>i</code> is $\leq$ pivot; everything from <code>i</code> to <code>r-1</code> is $>$ pivot; so slot <code>i</code> is exactly where the pivot belongs.
<code>return i</code>	The pivot's final index.

8.3.5 Manual trace: [4, 2, 7, 3, 1, 5], pivot = 5

`l = 0, r = 5, pivot = arr[5] = 5, i = 0.`

<code>j</code>	<code>arr[j]</code>	<code>≤ 5?</code>	Action	Array after
0	4	yes	<code>swap(0,0), i→1</code>	[4, 2, 7, 3, 1, 5]
1	2	yes	<code>swap(1,1), i→2</code>	[4, 2, 7, 3, 1, 5]
2	7	no	—	[4, 2, 7, 3, 1, 5]
3	3	yes	<code>swap(2,3), i→3</code>	[4, 2, 3, 7, 1, 5]
4	1	yes	<code>swap(3,4), i→4</code>	[4, 2, 3, 1, 7, 5]
loop ends			<code>swap(4,5)</code>	[4, 2, 3, 1, 5, 7]
return i = 4				

Check: `arr[4] == 5` is the pivot; [4, 2, 3, 1] on the left are all ≤ 5 ; [7] on the right is > 5 . The pivot is in its final sorted position — in the fully sorted array [1, 2, 3, 4, 5, 7], the 5 is indeed at index 4.



8.3.6 Complexity

One pass: $n - 1$ comparisons, at most n swaps $\Rightarrow \Theta(n)$ time, $O(1)$ space.

Common Mistake

`j <= r` — the pivot gets compared with itself, swapped left, and the final swap then puts the wrong element at the boundary.

Starting `i = 1 - 1` and pre-incrementing — a common textbook variant (`if (arr[j] <= pivot) i++; swap(arr, i, j);` with the final `swap(arr, i+1, r)` returning `i+1`). It is correct, but mixing the two variants (`i = 1` with `swap(arr, i+1, r)`) produces silent corruption. Pick one and be consistent.

Returning $i - 1$ or j — Lomuto returns the pivot's index; anything else breaks the recursive boundaries.

All-equal input: with $\leq$ every element is swapped into the small side, so the pivot ends at the far right and the split is $n - 1 : 0 \Rightarrow O(n^2)$. This is Lomuto's worst practical weakness and the reason 3-way partitioning (Chapter 10) exists.

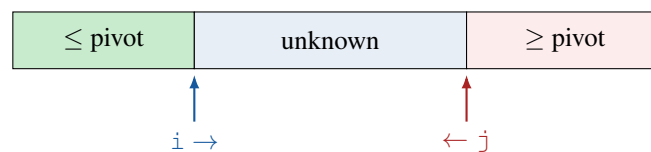
8.4 Hoare Partition

8.4.1 Intuition

Intuition

Instead of one scanning pointer, use two that walk towards each other. The left pointer advances while it sees elements that clearly belong on the left; the right pointer retreats while it sees elements that clearly belong on the right. When both stop, each is looking at an element that belongs on the *other* side — so swap them and continue. When the pointers meet or cross, every element has been placed on a correct side.

The pay-off is fewer swaps: Hoare only swaps genuinely misplaced pairs, whereas Lomuto swaps every small element it meets. On random data Hoare does about three times fewer swaps, which is why library implementations are Hoare-based.



8.4.2 Invariant

Invariant

Everything in $\text{arr}[l..i]$ is $\leq$ pivot; everything in $\text{arr}[j..r]$ is $\geq$ pivot; the region strictly between i and j is unexamined. The loop terminates when $i \geq j$, at which point j is the last index of the left group.

8.4.3 Java implementation

```

1  class Solution {
2
3      public static int hoarePartition(int[] arr, int l, int r) {
4          int pivot = arr[l];
5
6          int i = l - 1;
7          int j = r + 1;
8
9          while (true) {
10             do {
11                 i++;
12             } while (arr[i] < pivot);
13
14             do {
15                 j--;
16             } while (arr[j] > pivot);
17

```

```

18         if (i ≥ j)
19             return j;
20
21         swap(arr, i, j);
22     }
23 }
24 }

```

Listing 8.2: Hoare partition; pivot is the first element

8.4.4 Line by line

<code>int pivot = arr[l]</code>	The value, not the position. Unlike Lomuto, the pivot element is <i>not</i> parked out of the way — it participates in the scanning and may end up anywhere.
<code>i = l - 1, j = r + 1</code>	Deliberately one step <i>outside</i> the range, because each <code>do-while</code> increments/decrements <i>before</i> testing. Initialising to <code>l</code> and <code>r</code> would skip the first and last elements.
<code>do { i++; } while (arr[i] < pivot)</code>	Advance past everything strictly smaller; stop on an element $\geq$ pivot . A <code>do-while</code> guarantees at least one step, so the pointers always make progress and the loop cannot spin forever.
<code>do { j--; } while (arr[j] > pivot)</code>	Symmetrically, stop on an element $\leq$ pivot.
<code>if (i >= j) return j;</code>	The pointers met or crossed — done. Returning <code>i</code> instead of <code>j</code> is a classic bug that causes infinite recursion.
<code>swap(arr, i, j)</code>	Both pointers are on misplaced elements; one swap fixes two problems.
<code>no final pivot swap</code>	There is none, because the pivot has no designated home.

Why the strict comparisons matter. `arr[i] < pivot` stops *on* elements equal to the pivot. That looks wasteful (it causes a swap of two equal values) but it is essential: it forces both pointers to move, which prevents an infinite loop on arrays of equal elements. Changing them to `<=` and `>=` makes the pointers run off the end of the array.

8.4.5 Trace: [4, 2, 7, 3, 1, 5], pivot = arr[0] = 4

Step	i	j	Array
start	-1	6	[4, 2, 7, 3, 1, 5]
scan	0 (arr[0]=4 $\geq$ 4, stop)	4 (arr[4]=1 $\leq$ 4, stop)	
swap			[1, 2, 7, 3, 4, 5]
scan	2 (7 $\geq$ 4)	3 (3 $\leq$ 4)	
swap			[1, 2, 3, 7, 4, 5]
scan	3 (7 $\geq$ 4)	2 (3 $\leq$ 4)	
stop	3 $\geq$ 2	return j = 2	

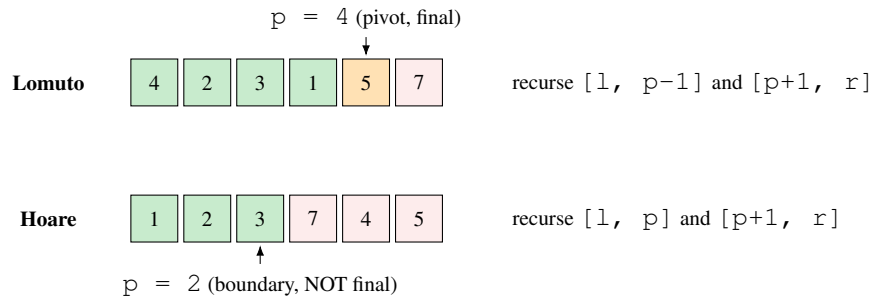
Result: [1, 2, 3 | 7, 4, 5]. The left group `arr[0..2]` is all $\leq$ 4; the right group `arr[3..5]` is all $\geq$ 4. But look: **the pivot value 4 sits at index 4**, not at the boundary index 2, and it is *not* in its final sorted position (sorted, the array is [1, 2, 3, 4, 5, 7], so 4 belongs at index 3).

8.5 The critical difference: what does the partition return?

Lomuto vs Hoare — the one thing you must not confuse

Lomuto returns the **pivot's final position**. `arr[p]` is done forever and is excluded from both recursive calls.

Hoare returns a **boundary index**, not a pivot position. It only guarantees that $\text{arr}[1..p] \leq \text{arr}[p+1..r]$. *No element is known to be final*, so `p` must be **included** in the left recursive call.



8.6 Recursive QuickSort

8.6.1 With Lomuto

```

1 public static void quickSort(int[] arr, int l, int r) {
2     if (l ≥ r)
3         return;
4
5     int p = lomutoPartition(arr, l, r);
6
7     quickSort(arr, l, p - 1);    // exclude the pivot: it is final
8     quickSort(arr, p + 1, r);
9 }

```

Listing 8.3: QuickSort with Lomuto partition

Call with `quickSort(arr, 0, arr.length - 1);`

8.6.2 With Hoare

```

1 public static void quickSortHoare(int[] arr, int l, int r) {
2     if (l ≥ r)
3         return;
4
5     int p = hoarePartition(arr, l, r);
6
7     quickSortHoare(arr, l, p);    // INCLUDE p: nothing is final
8     quickSortHoare(arr, p + 1, r);
9 }

```

Listing 8.4: QuickSort with Hoare partition

8.6.3 Why the ranges differ — carefully

	Lomuto	Hoare
Meaning of p	index where the pivot now lives, permanently	last index of the left group; the element there may still move
Left call	$[l, p-1]$	$[l, p]$
Right call	$[p+1, r]$	$[p+1, r]$
Total covered	$(p-l) + (r-p) = r-l$ elements, plus the pivot	$(p-l+1) + (r-p) = r-l+1$ elements — all of them

The rule is mechanical: *every element must appear in exactly one recursive call, unless it is provably in its final position*. Lomuto proves one element final per call, so it drops it. Hoare proves nothing final, so it must keep everything.

Common Mistake

Using `quickSortHoare(arr, l, p - 1)` with Hoare. The element at index p is then never sorted and simply left behind. Symptom: output almost sorted, with a few elements out of place. This is the single most common QuickSort bug.

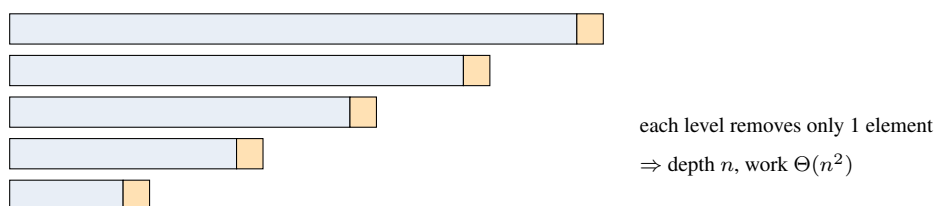
Using `quickSort(arr, l, p)` with Lomuto. The pivot is included in the left range again, so if the pivot lands at the far right ($p == r$, e.g. on sorted input), the recursive call is `quickSort(arr, l, r)` — **identical to the current call**. Infinite recursion, `StackOverflowError`.

Returning i from Hoare instead of j . With `quickSortHoare(arr, l, p)` and $p == i$, the same range can recur when $i == l$. Infinite recursion again.

Using `arr[r]` as the Hoare pivot with the standard loop. Hoare's version is written for `arr[l]`; using `arr[r]` makes i run past the end on sorted input. If you want a different pivot, swap it to position l first.

8.7 Worst case, and how to avoid it

QuickSort is $O(n^2)$ when every partition is maximally unbalanced — one side gets $n-1$ elements and the other gets 0. With `pivot = arr[r]` (Lomuto), that is exactly what happens on **already sorted** and on **reverse sorted** input:



Random pivot	Swap a randomly chosen index into the pivot position before partitioning. Expected $O(n \log n)$ on <i>every</i> input; an adversary cannot construct a bad case. Two lines of code, and the standard interview answer.
Median-of-three	Use the median of <code>arr[l]</code> , <code>arr[m]</code> , <code>arr[r]</code> . Cheap, and defeats the sorted-input case specifically.
Recurse into the smaller side first	Tail-call the larger side; bounds the stack at $O(\log n)$ even in the worst case.
3-way partitioning	Handles many duplicate keys, Lomuto's other weak spot (Chapter 10).

```

1 private static int randomizedPartition(int[] arr, int l, int r) {
2     int idx = l + (int) (Math.random() * (r - l + 1));
3     swap(arr, idx, r);           // move it to the pivot slot
4     return lomutoPartition(arr, l, r);
5 }

```

Listing 8.5: Randomised pivot — add this in an interview

8.8 Complexity summary

	Time	Reason
Best	$\Theta(n \log n)$	pivot splits evenly $\Rightarrow$ depth $\log n$, $\Theta(n)$ per level
Average	$\Theta(n \log n)$	random pivot gives expected constant-ratio splits
Worst	$\Theta(n^2)$	pivot is always min or max $\Rightarrow$ depth n
Space	$O(\log n)$ avg, $O(n)$ worst	recursion stack only; partitioning is in-place

Interview Tip

Volunteer the worst case before you are asked, then immediately give the fix: “This is $O(n^2)$ if the pivot is always extreme — for instance on sorted input with a last-element pivot — so in practice I’d randomise the pivot, which makes the expected time $O(n \log n)$ regardless of the input.” That single sentence covers what most candidates miss.

Practice

1. Trace Lomuto on $[1, 2, 3, 4, 5]$. What is p each time? How deep does the recursion go?
2. Trace Hoare on $[5, 5, 5, 5]$. Confirm the pointers still make progress and the split is balanced. Then trace Lomuto on the same input and see it degenerate.
3. Take the Hoare QuickSort and change the left call to $[1, p-1]$. Run it mentally on $[4, 2, 7, 3, 1, 5]$ and find which element ends up misplaced.

Chapter 9

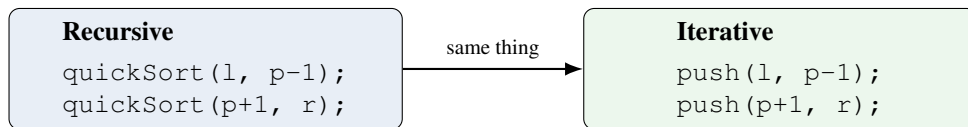
Iterative QuickSort and the Explicit Stack

9.1 What recursion actually is

A recursive call does three things: it saves the current local variables somewhere, jumps to the start of the function with new arguments, and remembers where to come back to. That “somewhere” is the call stack — a stack the runtime maintains for you.

Key Insight

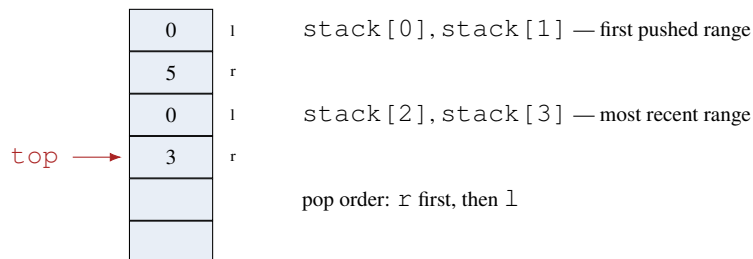
For QuickSort, the *only* thing a stack frame needs to remember is the pair (l, r) — the range still to be sorted. There is no combine step, so nothing has to happen after the recursive calls return. That is why QuickSort converts to an iterative form so cleanly: we can throw away the runtime’s stack and keep our own stack of ranges.



Notice this does *not* work as easily for Merge Sort: there, work must happen *after* the recursive calls (the merge), so a plain stack of ranges is not enough — you would need to record how far along each frame was. That is exactly why the iterative Merge Sort of Chapter 6 looks completely different: it reschedules the merges by width instead of simulating the stack.

9.2 The stack as a pair of ints

We do not need an object or a Deque. Push l , then r ; pop r , then l . Two `int` slots per pending range.



Key Insight

The pop order must **mirror** the push order. We push l then r , so a stack returns r first. Reversing this silently swaps the two bounds, giving $l > r$, which either does nothing (if you guard with $l \geq r$) or crashes.

9.3 Iterative QuickSort with Lomuto

```
1 class Solution {
2
3     public static void quickSortIterative(int[] arr) {
4         if (arr.length ≤ 1)
```

```

5         return;
6
7         int[] stack = new int[arr.length];    // see the note on sizing below
8         int top = -1;
9
10        stack[++top] = 0;                      // push l
11        stack[++top] = arr.length - 1;        // push r
12
13        while (top ≥ 0) {
14            int r = stack[top--];              // pop r FIRST
15            int l = stack[top--];              // then l
16
17            int p = lomutoPartition(arr, l, r);
18
19            if (p - 1 > l) {                    // left range has ≥ 2 elements
20                stack[++top] = l;
21                stack[++top] = p - 1;
22            }
23
24            if (p + 1 < r) {                    // right range has ≥ 2 elements
25                stack[++top] = p + 1;
26                stack[++top] = r;
27            }
28        }
29    }
30 }

```

Listing 9.1: Iterative QuickSort, Lomuto partition

9.3.1 Line by line

<code>int top = -1</code>	Empty-stack sentinel. <code>stack[++top] = x</code> is “increment then store” — push. <code>stack[top--]</code> is “read then decrement” — pop.
<code>push 0 then n-1</code>	The initial range, exactly the arguments of the first recursive call.
<code>while (top >= 0)</code>	Keep going while ranges remain. This loop is the recursion.
<code>pop r then l</code>	Mirror of the push order.
<code>lomutoPartition(arr, l, r)</code>	Unchanged — partitioning was always iterative.
<code>if (p - 1 > l)</code>	Push the left range only if it has at least two elements. This is the iterative equivalent of the recursive base case <code>if (l >= r) return;</code> — checking before pushing keeps the stack smaller.
<code>if (p + 1 < r)</code>	Same for the right range.

Key Insight

Because a stack is LIFO, the *last* range pushed is processed *first*. So pushing left then right means the **right** side is sorted first — the opposite order to the recursive version. That is completely fine: the two sides are independent, so any order produces the same sorted array. Only the sequence of intermediate states differs.

9.3.2 Trace on [4, 2, 7, 3, 1, 5]

#	Pop	Partition	Array	Stack after
1	—	—	[4, 2, 7, 3, 1, 5]	[(0, 5)]
2	(0, 5)	p=4	[4, 2, 3, 1, 5, 7]	[(0, 3)] (right (5, 5) too small)
3	(0, 3)	pivot 1, p=0	[1, 2, 3, 4]	[(1, 3)]
4	(1, 3)	pivot 4, p=3	[1, 2, 3, 4]	[(1, 2)]
5	(1, 2)	pivot 3, p=2	[1, 2, 3, 4]	[] (both sides size 1)
6	—	stack empty	[1, 2, 3, 4, 5, 7]	done

9.4 Iterative QuickSort with Hoare

The only changes are the recursive boundaries (as always) and an explicit guard, because Hoare's ranges overlap the returned index.

```

1  class Solution {
2
3      public static void quickSortHoareIterative(int[] arr) {
4          if (arr.length ≤ 1)
5              return;
6
7          int[] stack = new int[arr.length];
8          int top = -1;
9
10         stack[++top] = 0;
11         stack[++top] = arr.length - 1;
12
13         while (top ≥ 0) {
14             int r = stack[top--];
15             int l = stack[top--];
16
17             if (l ≥ r)
18                 continue;           // base case, checked on pop
19
20             int p = hoarePartition(arr, l, r);
21
22             if (p > l) {               // left range is [l, p]
23                 stack[++top] = l;
24                 stack[++top] = p;
25             }
26
27             if (p + 1 < r) {          // right range is [p+1, r]
28                 stack[++top] = p + 1;
29                 stack[++top] = r;
30             }
31         }
32     }
33 }
```

Listing 9.2: Iterative QuickSort, Hoare partition

Note `stack[++top] = p` (not `p - 1`) — exactly the same distinction as in the recursive version. If you can explain why this line differs between the two schemes, you understand partitioning.

9.5 How big must the stack be?

Each iteration pops one range and pushes at most two, so the stack grows by at most one range (two ints) per level of the conceptual recursion tree. In the worst case the depth is n , so a safe-but-large bound is $2n$ ints.

<code>new int[arr.length]</code>	Works in practice for balanced partitions ($O(\log n)$ depth), and is what most textbooks show — but it can overflow on adversarial input.
<code>new int[2 * arr.length]</code>	Always safe.
Better: push the larger side, loop on the smaller	Bounds the stack at $O(\log n)$ <i>provably</i> , because each pushed range is at most half the current one.

```

1  while (top ≥ 0) {
2      int r = stack[top--];
3      int l = stack[top--];
4
5      while (l < r) {
6          int p = lomutoPartition(arr, l, r);
7
8          if (p - l < r - p) {                // left side is smaller
9              if (p - l > 1) {                // push the LARGER side
10                 stack[++top] = p + 1;
11                 stack[++top] = r;
12             }
13             r = p - 1;                      // loop on the smaller side
14         } else {
15             if (p + 1 < r) {
16                 stack[++top] = l;
17                 stack[++top] = p - 1;
18             }
19             l = p + 1;
20         }
21     }
22 }

```

Listing 9.3: Bounded-stack variant: always recurse into the smaller side

This is the tail-call elimination trick: one of the two “recursive calls” becomes a loop iteration, so only the smaller half ever reaches the stack. Since a pushed range is at most half the size of the range it came from, the stack depth is at most $\log_2 n$.

9.6 Recursive vs iterative, across all five algorithms

Algorithm	Recursive	Iterative	Notes
Bubble Sort	possible but pointless	natural	nested loops, no subproblems
Selection Sort	possible but pointless	natural	same
Insertion Sort	possible but pointless	natural	same
Merge Sort	common (top-down)	bottom-up by width	work happens after the calls, so no simple stack simulation
QuickSort	common	explicit stack of ranges	no combine step, so ranges are all the state needed

9.6.1 When is recursion worth it?

Prefer recursion when	Prefer iteration when
The problem is genuinely defined on subproblems (divide and conquer, trees)	The problem is a straightforward sweep
Readability matters and depth is $O(\log n)$	Depth could be $O(n)$ and you risk <code>StackOverflowError</code>
The combine step needs values returned from below	There is nothing to do after the sub-calls
You are writing it on a whiteboard	You are in an embedded / constrained environment, or the interviewer explicitly asks for $O(1)$ auxiliary space

Key Insight

Iteration does not make the algorithm cheaper — QuickSort still needs $O(\log n)$ memory for the pending ranges. It moves that memory from the *call stack* (a fixed, limited resource, typically 512 KB–1 MB, and fatal to exhaust) to the *heap* (large, and you control the size). That is the real reason to convert.

Common Mistake

Popping 1 before `r` — bounds arrive swapped.

Using `stack[top++]` for push — stores at the old `top`, one slot too low, corrupting the previous entry. Push is pre-increment, pop is post-decrement.

`while (top > 0)` instead of `>= 0` — the final range is never processed, so the array is left partly unsorted.

Pushing ranges without the size guard, and having no base-case check on pop — ranges like $(3, 3)$ or $(4, 3)$ reach `partition`, which then reads out of bounds. Either guard on push (Lomuto version) or on pop (Hoare version); do at least one.

Sizing the stack as `new int[(int) Math.log(n)]` — ignores the worst case and throws `ArrayIndexOutOfBoundsException` on sorted input.

Practice

1. Convert the recursive Lomuto QuickSort to iterative from memory, then diff it against the listing above.
2. Add a counter for the maximum value `top` reaches. Run it on sorted input of size 20 with a last-element pivot, then with a random pivot. Compare.
3. Why can bottom-up Merge Sort not be written as a simple stack of ranges? (Answer: the merge must run *after* both sub-ranges are done, so a frame needs a “phase” field — essentially re-implementing the call stack properly.)

Chapter 10

Partitioning Beyond QuickSort

10.1 Partitioning is an idea, not a subroutine

We met partitioning as a step inside QuickSort. That undersells it. Partitioning is a general technique:

rearrange an array in one linear pass so that elements are grouped by a predicate.

Once you see it that way, a whole family of problems becomes the same problem.

Scheme	Groups	Typical use
Lomuto	2 ($\leq$, $>$)	QuickSort, Quickselect; simple to write
Hoare	2 ($\leq$, $\geq$)	QuickSort in real libraries; fewer swaps
2-way (general)	2 (predicate true/false)	move all evens left, all negatives left, stable-partition variants
3-way / Dutch National Flag	3 ($<$, $=$, $>$)	many duplicate keys; LeetCode 75 — Sort Colors

10.2 Sorting vs partitioning vs selection

This distinction is worth real attention, because choosing the weakest tool that solves the problem is often the entire point of an interview question.

Sorted

1	2	3	4	5	6
---	---	---	---	---	---

 total order — every element final

Partitioned

1	3	2	5	8	7	9
---	---	---	---	---	---	---

 only the pivot is final

Key Insight

Partitioning does not sort either side. Look at $[1, 3, 2 \mid 5 \mid 8, 7, 9]$: the left group is $[1, 3, 2]$, which is not sorted, and the right is $[8, 7, 9]$, also not sorted. All partitioning guarantees is *membership* — which side of the pivot each element belongs to. That is exactly why QuickSort must recurse after partitioning: without the recursive calls, you have grouping, not order.

	Guarantee	Cost	Example problem
Sorting	every element in final position	$O(n \log n)$	LeetCode 912 — Sort an Array
Partitioning	one element final; the rest grouped	$O(n)$ per pass	LeetCode 75 — Sort Colors
Selection	the k -th element final; others merely grouped	$O(n)$ expected	LeetCode 215 — Kth Largest Element

10.2.1 Selection: Quickselect

If you only want the k -th smallest element, you do not need both recursive calls — only the one containing index k . That turns $T(n) = 2T(n/2) + n$ into $T(n) = T(n/2) + n = O(n)$.

```

1  class Solution {
2
3      public static int quickSelect(int[] arr, int k) {          // k is 0-indexed
4          int l = 0, r = arr.length - 1;
5
6          while (l ≤ r) {
7              int p = lomutoPartition(arr, l, r);
8
9              if (p == k) return arr[p];          // the pivot IS the answer
10             else if (p < k) l = p + 1;          // answer is to the right
11             else r = p - 1;                      // answer is to the left
12         }
13         return -1;                                  // unreachable for valid k
14     }
15 }

```

Listing 10.1: Quickselect: k th smallest in expected $O(n)$

Key Insight

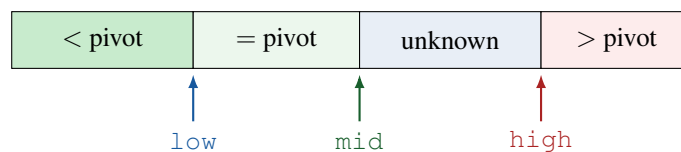
Quickselect is QuickSort with **one recursive call deleted**, and written as a loop because that call is a tail call. The geometric series $n + n/2 + n/4 + \dots = 2n$ gives $O(n)$ expected time — strictly better than sorting. **LeetCode 215 — Kth Largest Element in an Array** is exactly this, with $k = \text{arr.length} - k$.

10.3 Three-way partitioning — the Dutch National Flag

10.3.1 The problem with two-way partitioning

Give Lomuto an array of identical values. Every element satisfies $\text{arr}[j] \leq \text{pivot}$, so every one is swapped into the left group and the pivot lands at the far right: a $n - 1 : 0$ split, $O(n^2)$ overall. Duplicate-heavy data is common (categories, flags, grades, colours), so this matters.

The fix is to stop pretending there are two groups. Elements *equal* to the pivot are already in their final block and never need to be looked at again.



Invariant

- $\text{arr}[0 \dots \text{low}-1]$ are all $< \text{pivot}$ (all 0s);
- $\text{arr}[\text{low} \dots \text{mid}-1]$ are all $= \text{pivot}$ (all 1s);
- $\text{arr}[\text{mid} \dots \text{high}]$ is **unexamined**;
- $\text{arr}[\text{high}+1 \dots n-1]$ are all $> \text{pivot}$ (all 2s).

The loop runs while $\text{mid} \leq \text{high}$; when they cross, the unknown region is empty and the array is partitioned.

10.3.2 LeetCode 75 — Sort Colors

Problem. An array contains only 0s, 1s and 2s. Sort it in one pass, in place, without a library sort.

Intuition

Look at `arr[mid]` — the first unexamined element — and ask which of three homes it needs.

- **0**: it belongs in the left block. Swap it to `low`, then advance *both* `low` and `mid`.
- **1**: it already belongs where it is (the middle block ends at `mid`). Just advance `mid`.
- **2**: it belongs in the right block. Swap it to `high` and shrink `high` — but do **not** advance `mid`, because the value we just received from `high` has never been examined.

Key Insight

The asymmetry between the 0-case and the 2-case is the crux of this problem, and the thing interviewers probe. When we swap with `low`, the incoming element came from the region `[low, mid)`, which we know contains only 1s — so it is safe and `mid` can advance. When we swap with `high`, the incoming element came from the *unknown* region — so it must be examined, and `mid` must stay put.

```

1  class Solution {
2
3      public void sortColors(int[] nums) {
4          int low = 0;
5          int mid = 0;
6          int high = nums.length - 1;
7
8          while (mid ≤ high) {
9              if (nums[mid] == 0) {
10                 swap(nums, low, mid);
11                 low++;
12                 mid++;           // safe: incoming value is a 1
13             } else if (nums[mid] == 1) {
14                 mid++;           // already in the right block
15             } else {            // nums[mid] == 2
16                 swap(nums, mid, high);
17                 high--;          // do NOT advance mid: value unexamined
18             }
19         }
20     }
21
22     private void swap(int[] a, int i, int j) {
23         int t = a[i];
24         a[i] = a[j];
25         a[j] = t;
26     }
27 }

```

Listing 10.2: LeetCode 75 — Sort Colors, Dutch National Flag

10.3.3 Trace on [2, 0, 2, 1, 1, 0]

#	low	mid	high	nums[mid]	Action	Array
1	0	0	5	2	swap(0,5), high→4	[0, 0, 2, 1, 1, 2]
2	0	0	4	0	swap(0,0), low, mid→1	[0, 0, 2, 1, 1, 2]
3	1	1	4	0	swap(1,1), low, mid→2	[0, 0, 2, 1, 1, 2]
4	2	2	4	2	swap(2,4), high→3	[0, 0, 1, 1, 2, 2]
5	2	2	3	1	mid→3	[0, 0, 1, 1, 2, 2]
6	2	3	3	1	mid→4	[0, 0, 1, 1, 2, 2]
7	mid = 4 > high = 3 ⇒ stop					

Each iteration either advances `mid` or decrements `high`, so the gap `high - mid` strictly shrinks: at most n iterations. $\Theta(n)$ time, $O(1)$ space, single pass.

Common Mistake

Advancing `mid` in the `== 2` branch. The value swapped in from `high` is never examined, so a 0 or 2 can be stranded in the middle block. `[2, 0, 1]` then produces `[1, 0, 2]`. This is *the* bug for this problem.

while (`mid < high`) instead of `<=` — the last unexamined element is skipped. `[2, 1]` fails.

Counting sort instead (count 0s/1s/2s, then overwrite) — perfectly correct and $O(n)$, but it is two passes, and the problem explicitly asks for one. Mention it as an alternative, then give the DNF solution.

Forgetting to advance `low` in the `== 0` branch — the left block never grows, so 0s pile up incorrectly.

10.3.4 Three-way partitioning inside QuickSort

The same idea generalises from `{0,1,2}` to an arbitrary pivot value, and it repairs QuickSort's duplicate-key weakness:

```
1 public static void quickSort3Way(int[] arr, int l, int r) {
2     if (l ≥ r) return;
3
4     int pivot = arr[l + (r - l) / 2];
5     int low = l, mid = l, high = r;
6
7     while (mid ≤ high) {
8         if (arr[mid] < pivot) swap(arr, low++, mid++);
9         else if (arr[mid] > pivot) swap(arr, mid, high--);
10        else mid++;
11    }
12
13    quickSort3Way(arr, l, low - 1); // strictly less than pivot
14    quickSort3Way(arr, high + 1, r); // strictly greater than pivot
15    // the equal block [low, high] is already final - skipped entirely
16 }
```

Listing 10.3: 3-way QuickSort: $O(n)$ on arrays with few distinct values

Key Insight

The equal-block is excluded from *both* recursive calls. On an array with only k distinct values, the recursion depth is $O(\log k)$ rather than $O(\log n)$, and an array of all-identical values is sorted in a single $O(n)$ pass. This is why Java's `Arrays.sort(int[])` uses a dual-pivot scheme with equal-element handling.

10.4 General 2-way partitioning

Not every partition involves a pivot value. The same pointer discipline groups by any predicate:

```
1 public static int partitionByPredicate(int[] arr) {
2     int i = 0;
3     for (int j = 0; j < arr.length; j++) {
4         if (arr[j] % 2 == 0) { // predicate: "belongs on the left"
5             swap(arr, i, j);
6             i++;
7         }
8     }
9     return i; // start index of the "false" group
}
```

```
10 }
```

Listing 10.4: Move all even numbers to the front (unstable, one pass)

This is Lomuto with the comparison replaced by an arbitrary test — proof that the pivot was never essential. Problems that reduce to this shape include **LeetCode 283 — Move Zeroes**, **LeetCode 905 — Sort Array By Parity**, and “group all failing records to the end”.

Interview Tip

When a problem says “in place”, “one pass”, “ $O(1)$ space”, and “group / separate / move all X to one side”, it is a partitioning problem, not a sorting problem. Sorting would be $O(n \log n)$ and is very often the wrong answer, even though it produces a valid output.

Practice

1. Solve **LeetCode 75 — Sort Colors** from the invariant alone, without looking at the code.
2. Solve **LeetCode 215 — Kth Largest Element** with Quickselect. What is the expected time, and what is the worst case? How would you fix the worst case?
3. Modify the even/odd partition so that it is *stable*. Can it still be done in $O(1)$ space? (Answer: not easily — stability generally costs $O(n)$ extra space or $O(n \log n)$ time. Good interview discussion.)

Chapter 11

LeetCode Roadmap and Pattern Recognition

Practice

Solutions are deliberately *not* given here. For each problem, write down the four things — *invariant*, *state*, *pass structure*, *edge cases* — before you type anything. If you cannot, re-read the referenced chapter rather than the solution.

11.1 Beginner

912 — Sort an Array

Medium

Concept	Merge Sort or QuickSort, implemented by hand
Why it is useful	The canonical “prove you can implement $O(n \log n)$ sorting” problem. Library calls are disallowed in spirit, and the test data is deliberately adversarial.
Prerequisite	Chapters 5–8
Before coding	Which algorithm, and why? The test set includes large sorted and duplicate-heavy arrays, so plain Lomuto with a last-element pivot will TLE. Decide up front: Merge Sort (safe) or randomised/3-way QuickSort (fast).

147 — Insertion Sort List

Medium

Concept	Insertion Sort on a linked list
Why it is useful	Forces you to think about the <i>invariant</i> rather than array indices. There is no “shift” on a list — you relink instead.
Prerequisite	Section on Insertion Sort, Chapter 3
Before coding	What is the sorted prefix here? Where does the “key” live? Why is a dummy head node almost mandatory? What do you scan from — the head, or the current position?

75 — Sort Colors

Medium

Concept	Dutch National Flag / 3-way partitioning
Why it is useful	The purest partitioning problem there is, and a very common interview question.
Prerequisite	Chapter 10
Before coding	State the four regions and the three pointers. Then answer the key question: why does <code>mid</code> advance after a swap with <code>low</code> but not after a swap with <code>high</code> ?

11.2 Intermediate

148 — Sort List

Medium

Concept	Merge Sort on a linked list
Why it is useful	Merge Sort's natural home. The follow-up asks for $O(1)$ space, which rules out recursion and points at the bottom-up version.
Prerequisite	Chapters 5 and 6
Before coding	How do you find the middle of a list? (slow/fast pointers) How do you split it? (cut the link — do not forget, or you get infinite recursion) Then: how would the bottom-up version avoid finding the middle at all?

215 — Kth Largest Element in an Array

Medium

Concept	Quickselect (partition-based selection); heap as an alternative
Why it is useful	The clearest demonstration that <i>selection is cheaper than sorting</i> : $O(n)$ expected vs $O(n \log n)$.
Prerequisite	Chapters 8 and 10
Before coding	Convert “ k -th largest” to a 0-indexed position. Which side do you recurse into after partitioning, and why only one? What is the worst case, and what one-line change fixes it? Also be ready to compare with a size- k min-heap ($O(n \log k)$) and say when each wins.

493 — Reverse Pairs

Hard

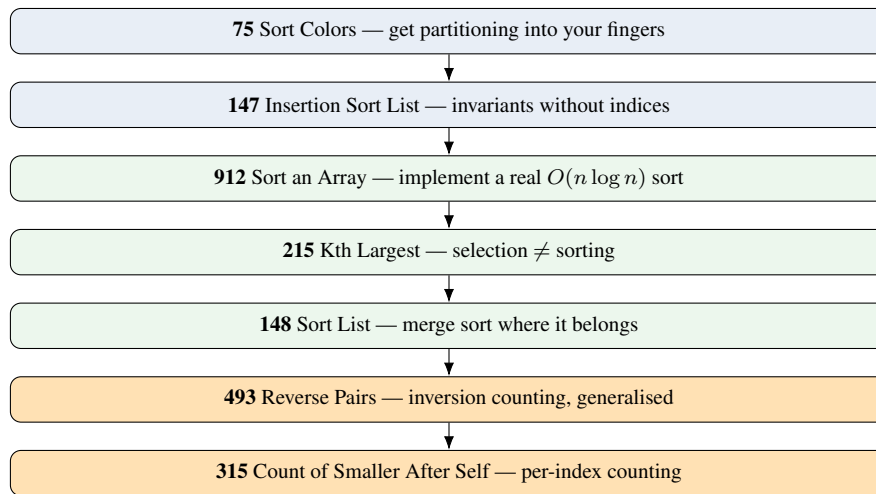
Concept	Inversion counting with a modified condition ($a[i] > 2 * a[j]$)
Why it is useful	Shows that the merge-sort counting technique is a <i>framework</i> , not a single trick.
Prerequisite	Chapter 7
Before coding	The counting condition no longer matches the merging comparison, so you cannot fold them into one loop. Where does the extra counting pass go, and why is it still $O(n)$ per merge? Watch for overflow — use <code>long</code> for $2 * a[j]$ and for the answer.

315 — Count of Smaller Numbers After Self

Hard

Concept	Per-element inversion counting; alternatives via BIT / merge sort on indices
Why it is useful	The hardest and most instructive member of the inversion family: you need a count <i>per index</i> , not a total.
Prerequisite	Chapter 7
Before coding	If you sort the values, you lose track of which original index each belongs to. What should you sort instead? (Hint: an array of indices, comparing by value.) Where in the merge do you add to a particular element's counter?

11.3 Suggested order

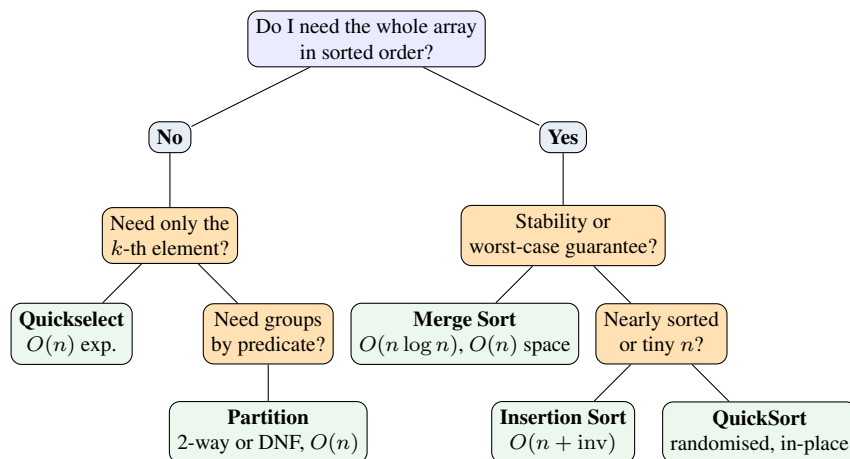


11.4 How do I know which sorting technique to use?

Ask these ten questions, roughly in order. The first one that gets a decisive answer usually settles it.

Question	If yes
1 Do I actually need the <i>whole</i> array sorted?	If not, consider partitioning ($O(n)$), selection ($O(n)$), or a heap ($O(n \log k)$).
2 Is the input nearly sorted?	Insertion Sort: $O(n + \text{inversions})$. Also why TimSort exists.
3 Do I need stability?	Merge Sort / TimSort. Never QuickSort, never Selection Sort.
4 Do I need $O(1)$ extra space?	QuickSort, heapsort, or in-place partitioning — not standard Merge Sort.
5 Do I need to partition around a pivot?	Lomuto or Hoare directly; no full sort.
6 Do I only need the k -th smallest/largest?	Quickselect, $O(n)$ expected.
7 Am I counting pairs (i, j) with a reversed order condition?	Inversion counting via merge sort, $O(n \log n)$.
8 Is the input constrained to a small range of values?	Counting sort $O(n + k)$, or 3-way partitioning if there are very few distinct values.
9 Can I exploit frequency instead of order?	A hash map or bucket array — e.g. “top k frequent” needs counting, not sorting.
10 Does sorting merely <i>enable</i> a two-pointer sweep?	Then the sort is preprocessing; use the library sort and spend your thinking on the sweep (3Sum, merge intervals, meeting rooms).

11.4.1 Decision tree

**Key Insight**

Two questions carry most of the weight in practice: (1) *do I need the entire order, or just one element / a grouping?* and (2) *do I need stability or a worst-case guarantee?* Question 1 chooses between $O(n)$ and $O(n \log n)$; question 2 chooses between Merge Sort and QuickSort. Everything else is refinement.

Interview Tip

In a real interview, when sorting is only a preprocessing step, say so and use `Arrays.sort()`. Hand-rolling Merge Sort for a 3Sum problem wastes ten minutes and signals that you cannot tell the core of a problem from its scaffolding. Implement sorting by hand only when the question is *about* sorting.

Chapter 12

Common Bugs and How to Find Them

12.1 Symptom table

Symptom	Look at section
<code>StackOverflowError</code>	12.9, 12.8
<code>ArrayIndexOutOfBoundsException</code>	12.2, 12.10, 12.6
Output almost sorted, a few elements wrong	12.8, 12.4
Zeros appear in the output	12.5
Duplicates lost or multiplied	12.4, 12.5
Correct but times out	12.7, pivot choice (Ch. 8)
Wrong only for large inputs	12.3
Multi-key sort loses the secondary order	12.11
Bottom-up leaves the tail unsorted	12.6, 12.7

12.2 Off-by-one in loop bounds

```
for (int j = 0; j ≤ arr.length - 1 - i; j++)  
    if (arr[j] > arr[j + 1]) ...           // reads arr[arr.length] on i = 0
```

Listing 12.1: Bubble Sort — broken

On $[1, 2]$: $i = 0$, j reaches 1, and `arr[2]` throws.

```
for (int j = 0; j < arr.length - 1 - i; j++)
```

Listing 12.2: Fixed

Rule: whenever the body reads `arr[j+1]`, the bound must be $j < \text{length} - 1$, and any $-i$ comes on top of that.

12.3 The `mid` calculation

```
int m = (l + r) / 2;
```

Listing 12.3: Broken

For $l = 2,000,000,000$ and $r = 2,100,000,000$ the sum exceeds $2^{31} - 1$ and wraps negative, so `m` is negative and the next array access throws.

```
int m = l + (r - l) / 2;
```

Listing 12.4: Fixed

$r - l$ is a non-negative difference that always fits, so no intermediate value overflows. This exact bug lived in `java.util.Arrays.binarySearch` from 1997 to 2006.

12.4 Incorrect merge boundaries

```
mergeSort(arr, l, m);
mergeSort(arr, m, r);           // 'm' is in BOTH halves
```

Listing 12.5: Broken — overlapping halves

On $[2, 1]$: $l = 0, r = 1, m = 0$. The second call is `mergeSort(arr, 0, 1)` — the same range we are in. Infinite recursion. Even when it terminates, the element at m is duplicated and another is lost.

```
mergeSort(arr, l, m);
mergeSort(arr, m + 1, r);
```

Listing 12.6: Fixed

Rule: the two sub-ranges must *partition* the parent — no overlap, no gap. Check by summing their lengths: $(m - l + 1) + (r - m) = r - l + 1$. ✓

A second variant of the same bug:

```
for (int x = 0; x < temp.length; x++)
    arr[x] = temp[x];           // should be arr[l + x]
```

Listing 12.7: Broken — copy-back forgets the offset

This writes every merged block to the front of the array. The result contains duplicated values and the tail is never touched.

12.5 Forgetting the drain loops

```
while (i ≤ m && j ≤ r) { ... }
// missing: while (i ≤ m) temp[k++] = arr[i++];
// missing: while (j ≤ r) temp[k++] = arr[j++];
for (int x = 0; x < temp.length; x++) arr[l + x] = temp[x];
```

Listing 12.8: Broken

Merging $[1, 2]$ with $[3, 4]$: the main loop stops as soon as $j > r$... actually it stops when the *right* side is exhausted, leaving 3 and 4 unwritten. Java initialises `int[]` to zero, so `temp` is $[1, 2, 0, 0]$ and the array becomes $[1, 2, 0, 0]$ — values silently replaced by zeros.

Rule: exactly one drain loop ever executes, but you must write both, because you do not know in advance which side runs out first.

12.6 The incomplete final block in bottom-up Merge Sort

```
int mid    = left + width - 1;
int right  = left + 2 * width - 1;
merge(arr, left, mid, right);
```

Listing 12.9: Broken

With $n = 7$, $width=4$, $left=4$: $right = 11$, and `merge` reads `arr[11]` ⇒ exception.

```
int mid    = Math.min(left + width - 1,    n - 1);
int right  = Math.min(left + 2 * width - 1, n - 1);
if (mid < right)
    merge(arr, left, mid, right);
```

Listing 12.10: Fixed

The guard also matters: with $n = 7$, $width=2$, $left=6$ we get $mid = right = 6$, a lone block with no partner. Merging it with an empty range is pointless; the guard skips it and a later, wider pass absorbs it.

12.7 `width *= 2` VS `width++`

```
for (int width = 1; width < n; width++)
```

Listing 12.11: Broken

After the `width-1` pass the array consists of sorted blocks of size 2. The next pass with `width = 2` correctly merges them into blocks of 4. But then `width = 3` assumes blocks of size 3, which do not exist — it merges `arr[0..2]` with `arr[3..5]`, two ranges that are *not* individually sorted. The merge produces garbage.

On `[5, 4, 3, 2, 1, 0]` this yields `[3, 2, 1, 0, 5, 4]`-style output: not sorted, no exception, no warning.

Rule: the `width` sequence must match what the previous pass produced, and the previous pass doubles. `width *= 2` is a correctness requirement, not a speed-up.

12.8 Lomuto vs Hoare recursive boundaries

```
int p = hoarePartition(arr, l, r);
quickSortHoare(arr, l, p - 1);      // WRONG: element p is dropped
quickSortHoare(arr, p + 1, r);
```

Listing 12.12: Broken — Hoare with Lomuto's boundaries

Hoare returns a *boundary*, not a final position. Index `p` is never included in any recursive call, so `arr[p]` is never sorted.

On `[4, 2, 7, 3, 1, 5]`: Hoare returns `p = 2` with `[1, 2, 3, 7, 4, 5]`. The left call handles `[0, 1]`, the right handles `[3, 5]`, and index 2 is skipped — here it happens to be correct, but on `[3, 1, 2]` the same bug leaves an element misplaced.

```
quickSortHoare(arr, l, p);
quickSortHoare(arr, p + 1, r);
```

Listing 12.13: Fixed

The mirror-image bug:

```
int p = lomutoPartition(arr, l, r);
quickSort(arr, l, p);      // pivot included again
```

Listing 12.14: Broken — Lomuto with Hoare's boundaries

On sorted input the pivot lands at `p == r`, so the call becomes `quickSort(arr, l, r)` — identical to the current one. Infinite recursion.

Key Insight

Memorise the pairing, not the numbers: **Lomuto returns a pivot position, so exclude it** (`p-1, p+1`); **Hoare returns a boundary, so include it** (`p, p+1`).

12.9 Infinite recursion in QuickSort

Three distinct causes, all producing `StackOverflowError`:

A recursive range equals the parent range	Lomuto with <code>[l, p]</code> , or Hoare returning <code>i</code> instead of <code>j</code> when <code>i == l</code> .
Missing base case	<code>if (l >= r) return;</code> omitted, or written as <code>if (l == r)</code> , which misses the empty case <code>l > r</code> .
Partition returns an out-of-range index	Typically from a mismatched <code>i = l</code> vs <code>i = l - 1</code> convention in Lomuto.

Diagnostic: print `(l, r)` at the top of the function. If the same pair appears twice in a row, you have found it immediately.

12.10 Incorrect `i/j` initialisation in Hoare

```
int i = l;
int j = r;
while (true) {
    do { i++; } while (arr[i] < pivot); // skips arr[l] entirely
    do { j--; } while (arr[j] > pivot); // skips arr[r] entirely
    ...
}
```

Listing 12.15: Broken

Because the loops are `do-while`, they move *before* testing. Starting at `l` and `r` means the first and last elements are never examined. The correct initialisation is deliberately *outside* the array:

```
int i = l - 1;
int j = r + 1;
```

Listing 12.16: Fixed

This looks alarming, but `i` is incremented before any access, so `arr[-1]` is never read.

A related trap: changing `arr[i] < pivot` to `arr[i] <= pivot`. On an array of equal values, `i` then runs past `r` and off the end of the array. The strict comparisons are what force both pointers to stop on equal elements and guarantee progress.

12.11 `<` vs `<=`: stability and duplicates

Place	Correct	Effect of getting it wrong
Merge comparison	<code>arr[i] <= arr[j]</code>	<code><</code> takes from the right on ties $\Rightarrow$ unstable . Values are still sorted, so tests on <code>int[]</code> pass and the bug only appears when sorting objects by a secondary key.
Inversion counting	<code>arr[i] <= arr[j]</code>	<code><</code> counts equal pairs as inversions. <code>[1, 1]</code> reports 1 instead of 0.
Insertion Sort inner loop	<code>arr[j] > key</code>	<code>>=</code> slides the key past equal elements: unstable, <i>and</i> an all-equal array degrades from $O(n)$ to $O(n^2)$.
Bubble Sort swap test	<code>arr[j] > arr[j+1]</code>	<code>>=</code> swaps equal neighbours: unstable, and the swapped flag never clears, so the early exit dies.
Lomuto comparison	<code>arr[j] <= pivot</code>	<code><</code> sends equal elements right; either choice degrades on all-equal input, which is why 3-way partitioning exists.

Common Mistake

Stability bugs are the most dangerous ones in this chapter because **they do not show up when sorting `int[]`**. Two equal ints are indistinguishable. The bug only surfaces when someone sorts `Employee[]` by salary after sorting by name, and the name order mysteriously scrambles. Get the comparison right

the first time.

12.12 A debugging routine

Key Insight

1. **Shrink the input.** Almost every bug here reproduces on 2–6 elements. Debug on $[2, 1]$, $[1, 2]$, $[2, 2]$, $[], [5]$.
2. **Test the four adversarial shapes:** already sorted, reverse sorted, all-equal, single element. These four catch the great majority of sorting bugs.
3. **Print the range.** For any recursive routine, log (l, r) on entry. A repeated pair means infinite recursion; a skipped index means a boundary bug.
4. **Check the invariant, not the output.** After a partition, assert that everything left of the boundary is $\leq$ everything right of it. After a merge pass, assert each block is sorted. The assertion fails at the *first* broken step rather than at the end.
5. **Count elements.** If the multiset of values changed, you have an overlap/copy-back bug, not an ordering bug.

```

1 public static void check(int[] original) {
2     int[] a = original.clone();
3     int[] b = original.clone();
4
5     mySort(a);
6     java.util.Arrays.sort(b);
7
8     if (!java.util.Arrays.equals(a, b))
9         throw new AssertionError("failed on "
10             + java.util.Arrays.toString(original));
11 }
12
13 // then:
14 check(new int[]{});
15 check(new int[]{5});
16 check(new int[]{2, 1});
17 check(new int[]{1, 2, 3, 4, 5}); // sorted: QuickSort worst case
18 check(new int[]{5, 4, 3, 2, 1}); // reverse
19 check(new int[]{7, 7, 7, 7, 7}); // all equal: Lomuto worst case
20 check(new int[]{7, 3, 5, 1, 9, 2, 6}); // odd size: bottom-up boundaries

```

Listing 12.17: A test harness worth writing once

Chapter 13

Master Cheat Sheet

13.1 The five mechanisms

EVERYTHING IN THIS BOOKLET, IN FIVE LINES

Bubble	adjacent comparison $\rightarrow$ swap
Selection	find minimum $\rightarrow$ swap into place
Insertion	key $\rightarrow$ shift larger elements right $\rightarrow$ insert
Merge	two sorted ranges $\rightarrow$ one sorted range
Partition	pivot $\rightarrow$ rearrange around it
Merge Sort	= Merge + recursion or Merge + doubling block width
QuickSort	= Partition + recursion or Partition + explicit stack

13.2 Quick reference

Bubble

```
for i in 0..n-2:
    swapped = false
    for j in 0..n-2-i:          // suffix of length i is final
        if arr[j] > arr[j+1]: swap; swapped = true
    if !swapped: break         // adaptivity
```

Invariant: after pass i , the largest $i + 1$ elements are at the end, final.

Selection

```
for i in 0..n-2:
    minIdx = i
    for j in i+1..n-1: if arr[j] < arr[minIdx]: minIdx = j
    swap(i, minIdx)           // exactly one swap per pass
```

Invariant: after iteration i , $\text{arr}[0..i]$ holds the $i + 1$ smallest, final.

Insertion

```
for i in 1..n-1:
    key = arr[i]; j = i-1
    while j ≥ 0 && arr[j] > key: arr[j+1] = arr[j]; j--
    arr[j+1] = key
```

Invariant: $\text{arr}[0..i-1]$ sorted (but not necessarily final). Cost = $\Theta(n + \text{inversions})$.

Merge

```

i = 1; j = m+1; k = 0
while i ≤ m && j ≤ r:
    temp[k++] = (arr[i] ≤ arr[j]) ? arr[i++] : arr[j++]    // ≤ keeps stability
while i ≤ m: temp[k++] = arr[i++]                        // drain
while j ≤ r: temp[k++] = arr[j++]                        // drain
copy temp back with offset 1

```

Top-down Merge Sort

```

if l ≥ r: return
m = 1 + (r-l)/2                // overflow-safe
mergeSort(l, m); mergeSort(m+1, r); merge(l, m, r)

```

Bottom-up Merge Sort

```

for width = 1; width < n; width *= 2:                // NOT width++
    for left = 0; left < n; left += 2*width:
        mid = min(left + width - 1, n-1)
        right = min(left + 2*width - 1, n-1)
        if mid < right: merge(left, mid, right)

```

Lomuto

```

pivot = arr[r]; i = l
for j in l..r-1: if arr[j] ≤ pivot: swap(i, j); i++
swap(i, r); return i                // i = PIVOT'S FINAL POSITION

```

[≤ pivot | > pivot | unknown | pivot] — one boundary (i), one scanner (j). Recurse [l, p-1] and [p+1, r].

Hoare

```

pivot = arr[l]; i = l-1; j = r+1
loop:
    do i++ while arr[i] < pivot
    do j-- while arr[j] > pivot
    if i ≥ j: return j                // j = BOUNDARY, not a final position
    swap(i, j)

```

[≤ pivot | unknown | ≥ pivot] — two pointers moving inward. Recurse [l, p] and [p+1, r].

Iterative QuickSort

```

push(0); push(n-1)
while stack not empty:
    r = pop(); l = pop()                // pop mirrors push order
    p = partition(l, r)
    if left range ≥ 2 elements: push(l); push(p-1)
    if right range ≥ 2 elements: push(p+1); push(r)

```

Dutch National Flag

```
low = 0; mid = 0; high = n-1
while mid ≤ high:
    if a[mid] == 0: swap(low++, mid++) // incoming is a known 1 → advance mid
    elif a[mid] == 1: mid++
    else: swap(mid, high--) // incoming unknown → do NOT advance mid
```

[0s | 1s | unknown | 2s]

Inversion

$$i < j \wedge \text{arr}[i] > \text{arr}[j]$$

Max = $\frac{n(n-1)}{2}$ (reverse sorted). Average for a random permutation $\approx n^2/4$. Insertion Sort cost = $\Theta(n + \text{inv})$. Counted during merge with `count += (m - i + 1)` whenever the right element is taken.

13.3 Complexity table

Algorithm	Best	Average	Worst	Space	Stable	In-place	Adaptive
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes	Yes
Merge Sort (TD)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No	No
Merge Sort (BU)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No	No
QuickSort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes	No
Quickselect	$O(n)$	$O(n)$	$O(n^2)$	$O(1)$	No	Yes	No
Partition (1 pass)	$O(n)$	$O(n)$	$O(n)$	$O(1)$	No	Yes	—
Count inversions	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	—	No	No

13.4 The distinctions that get tested

Lomuto returns	the pivot's final index	recurse [1, p-1], [p+1, r]
Hoare returns	a boundary	recurse [1, p], [p+1, r]
Merge Sort	divides blindly, combines cleverly	work on the way <i>up</i>
QuickSort	divides cleverly, combines trivially	work on the way <i>down</i>
Sorting	every element final	$O(n \log n)$
Partitioning	grouping only; one element final	$O(n)$
Selection	the k -th element final	$O(n)$ expected
width *= 2	matches what the last pass produced	correctness
width++	assumes blocks that do not exist	wrong output

13.5 Can You Reconstruct It?

SELF-TEST — answer out loud, then check the chapter

Elementary sorts

- ☐ What does Bubble Sort guarantee after one pass? (Ch.3)
- ☐ What does Selection Sort guarantee after iteration i ?
- ☐ What is the Insertion Sort invariant — and why does it say “sorted” but not “final”?
- ☐ Why is Insertion Sort $O(n)$ on a sorted array but Selection Sort is not?
- ☐ Which of the three are stable, and what breaks stability?
- ☐ Why does Selection Sort do only $\Theta(n)$ swaps, and when does that matter?

Inversions

- ☐ What does an inversion represent? (Ch.4)
- ☐ What is the maximum inversion count, and for which array?
- ☐ Why is Insertion Sort’s running time $\Theta(n + \text{inversions})$?
- ☐ Why does any adjacent-swap-only sort need $\Omega(n^2)$ in the worst case?

Merge Sort

- ☐ Why can two sorted halves be merged in linear time? (Ch.5)
- ☐ Where do the n and the $\log n$ in $O(n \log n)$ each come from?
- ☐ Why is $m = 1 + (r-1)/2$ preferred over $(1+r)/2$?
- ☐ Why must the right half start at $m+1$?
- ☐ Which comparison keeps Merge Sort stable, and why?
- ☐ How does bottom-up Merge Sort replace recursion? (Ch.6)
- ☐ Why `width *= 2` and not `width++`?
- ☐ Why `width < n` and not `width < n - 1`?
- ☐ What happens to a block with no partner?

Counting inversions

- ☐ What are the three disjoint groups of inversions? (Ch.7)
- ☐ How does the merge count cross inversions in one comparison?
- ☐ Why is the increment $(m - i + 1)$ and not $(m - i)$ or $(r - j + 1)$?
- ☐ Why must the return type be `long`?

QuickSort and partitioning

- ☐ What exactly does Lomuto return? (Ch.8)
- ☐ What exactly does Hoare return?
- ☐ Why are Hoare’s recursive boundaries different from Lomuto’s?
- ☐ Why are Hoare’s pointers initialised *outside* the range?
- ☐ Why must Hoare use strict `<` and `>` in its inner loops?
- ☐ Why does QuickSort become $O(n^2)$ in the worst case, and which input triggers it with a last-element pivot?
- ☐ Name two fixes for the worst case.
- ☐ Why does Lomuto degrade on an all-equal array, and what repairs it?

Iterative and structural

- ☐ How does an explicit stack replace recursive QuickSort? (Ch.9)
- ☐ Why is QuickSort easy to convert but Merge Sort is not?
- ☐ Why must the pop order mirror the push order?
- ☐ How do you bound the explicit stack at $O(\log n)$?

Partitioning and choice

- ☐ Why does partitioning *not* sort either side? (Ch.10)
- ☐ In Dutch National Flag, why does `mid` advance after a low swap but not after a high swap?
- ☐ How is Quickselect $O(n)$ when QuickSort is $O(n \log n)$?
- ☐ When would you choose Merge Sort over QuickSort? (Ch.11)
- ☐ Which four adversarial inputs should you always test?

Interview Tip

If you can answer every box above without looking, you can derive every algorithm in this booklet from scratch. That is a far more durable state than remembering thirteen implementations — and it is what actually holds up when an interviewer changes the problem halfway through.